



Министерство науки и высшего образования Российской Федерации
Мытищинский филиал
Федерального государственного автономного образовательного
учреждения
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Космический»

КАФЕДРА «Прикладная математика, информатика и вычислительная техника»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Разработка веб-приложения для отслеживания
привлекательности инвестиций

Студент КЗ-86Б

(Подпись, дата)

Я. А. Теплов
(И.О.Фамилия)

Руководитель ВКР

(Подпись, дата)

В.В. Афанасьева
(И.О.Фамилия)

Нормоконтролер

(Подпись, дата)

А.В. Чернышов
(И.О.Фамилия)

2026 г.

АННОТАЦИЯ

Расчетно-пояснительная записка состоит из 140 страниц, содержит 11 иллюстраций и 18 использованных источников.

Работа состоит из введения, двух глав и заключения

Цель работы состоит в разработке веб-приложения для мониторинга акций российских компаний и анализа их финансовых показателей.

В ходе выполнения работы были поставлены и решены следующие задачи:

- Разработка веб-приложения для отображения рыночных данных и финансовых показателей компаний.
- Реализация автоматизированного сбора и хранения рыночных данных по акциям российских компаний.
- Проектирование и разработка базы данных для хранения исторических рыночных и финансовых данных.
- Реализация расчета инвестиционных мультипликаторов P/E, P/B, P/S, Net Debt/E, FCF/E и показателя ROE на основе финансовой отчетности компаний.
- Реализация построения графиков стоимости акций и инвестиционных мультипликаторов.
- Разработка механизма сравнения нескольких акций на одном графике по выбранным показателям.
- Реализация системы пользовательских подписок на уведомления о достижении заданных значений цены акции и финансовых мультипликаторов.
- Реализация автоматической отправки уведомлений пользователям по электронной почте.
- Проведение тестирования разработанного программного обеспечения.

Содержание

ВВЕДЕНИЕ	4
1 Общетеchnическое обоснование	6
1.1 Цель работы	6
1.2 Постановка задачи	7
1.3 Актуальность разработки	8
1.4 Аналоги программы	9
2 Решение задачи.....	12
2.1 Выбор среды программирования	12
2.2 Выбор языка программирования	12
2.3 Выбор СУБД	13
2.4 Выбор системы построения интерфейса	14
2.5 Проектирование базы данных	15
2.6 Разработка программы	19
2.6.1 Выбранные библиотеки.....	19
2.6.2 Итоговая версия программы	20
2.6.3 Дополнительные функции	46
2.7 Тестирование программы	54
ЗАКЛЮЧЕНИЕ.....	57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	59
ПРИЛОЖЕНИЕ А	61
ПРИЛОЖЕНИЕ Б	69

ВВЕДЕНИЕ

Фондовый рынок предоставляет инвесторам возможность вложения денежных средств в акции компаний с целью получения дохода за счет роста их стоимости и дивидендных выплат. Для принятия обоснованных инвестиционных решений необходимо проводить анализ большого объема информации, включающей как рыночные данные, так и финансовые показатели компаний. Особое значение такой анализ имеет для долгосрочных инвесторов, ориентированных не на краткосрочные колебания цен, а на оценку реального финансового состояния бизнеса и его перспектив развития.

На практике получение необходимой информации связано с рядом трудностей. Данные о стоимости акций, финансовой отчетности компаний, дивидендах и инвестиционных показателях зачастую распределены между различными информационными ресурсами. В результате инвестору приходится использовать несколько сервисов одновременно, сопоставлять информацию из разных источников и самостоятельно отслеживать изменения интересующих показателей. Кроме того, большинство существующих инвестиционных платформ ориентировано преимущественно на совершение торговых операций и оперативное отслеживание рыночной ситуации. При этом возможности фундаментального анализа, как правило, представлены ограниченным набором инструментов или требуют использования сторонних сервисов.

Отдельную проблему представляет необходимость постоянного контроля инвестиционных показателей. Инвестору приходится регулярно проверять стоимость акций, значения коэффициентов P/E и P/B, а также другие характеристики компаний, чтобы своевременно реагировать на изменения рыночной ситуации. При анализе нескольких эмитентов одновременно такой процесс становится трудоемким и требует значительных временных затрат.

Для решения указанных проблем разрабатывалось мое веб-приложение, предназначенное для мониторинга акций российских компаний и анализа их финансовых показателей. Приложение объединяет данные о стоимости акций, исторических котировках, финансовой отчетности компаний, дивидендных

выплатах и инвестиционных коэффициентах в рамках единой информационной системы. Такой подход позволяет централизованно хранить информацию и предоставлять пользователю доступ к необходимым данным через единый интерфейс.

Разработанное приложение обеспечивает отображение исторических данных о стоимости акций и финансовых показателях компаний в графическом виде. Пользователь может анализировать динамику изменения цены, коэффициентов P/E и P/B, а также сравнивать несколько компаний на одном графике. Для повышения удобства мониторинга реализована система пользовательских подписок, позволяющая автоматически получать уведомления при достижении заданных значений цены акции или инвестиционных коэффициентов.

Таким образом, разработка программного средства, объединяющего инструменты визуализации, анализа и автоматического мониторинга инвестиционных показателей, является актуальной задачей и позволяет повысить удобство проведения фундаментального анализа для частных инвесторов.

1 Общетеchnическое обоснование

1.1 Цель работы

Целью данной выпускной квалификационной работы является разработка веб-приложения для мониторинга акций российских компаний и анализа их финансовых показателей, обеспечивающего централизованное хранение рыночных данных, расчет инвестиционных коэффициентов, визуализацию информации и автоматическое оповещение пользователей о достижении заданных условий.

Для достижения поставленной цели были определены следующие задачи:

1. Спроектировать и реализовать структуру базы данных для хранения информации об акциях, исторических котировках, финансовой отчетности компаний, дивидендных выплатах и пользовательских подписках.
2. Разработать механизм автоматизированного сбора и обновления рыночных данных по акциям российских компаний.
3. Реализовать расчет инвестиционных коэффициентов P/E, P/B, P/S, ROE, Net Debt/E, FCF/E на основе финансовой отчетности компаний.
4. Разработать веб-интерфейс для отображения рыночных и финансовых данных.
5. Реализовать построение графиков стоимости акций и инвестиционных мультипликаторов с возможностью выбора периода отображения данных.
6. Реализовать механизм сравнения нескольких акций на одном графике по выбранным показателям.
7. Разработать систему пользовательских подписок на уведомления о достижении заданных значений цены акции и инвестиционных коэффициентов.
8. Реализовать автоматическую отправку уведомлений пользователям по электронной почте.
9. Провести тестирование разработанного программного обеспечения и подтвердить корректность работы его основных функций.

1.2 Постановка задачи

В рамках выполнения выпускной квалификационной работы мной были спроектированы и реализованы следующие компоненты системы:

- Разработана структура реляционной базы данных для хранения информации об акциях российских компаний, исторических котировках, финансовой отчетности, дивидендных выплатах и пользовательских подписках на уведомления.
- Реализован модуль автоматизированного сбора рыночных данных с последующим сохранением информации в базе данных.
- Реализован механизм хранения исторических данных, позволяющий накапливать информацию о динамике стоимости акций и финансовых показателей компаний.
- Разработан модуль расчета инвестиционных коэффициентов P/E, P/B, P/S, ROE, Net Debt/E, FCF/E на основе данных финансовой отчетности.
- Создано веб-приложение с графическим интерфейсом, обеспечивающее отображение рыночных и финансовых данных в удобной для пользователя форме.
- Реализован механизм визуализации данных в виде графиков с возможностью выбора периода отображения и сравнения нескольких акций по выбранным показателям.
- Разработана система пользовательских подписок на уведомления о достижении заданных значений цены акции и инвестиционных коэффициентов.
- Реализован механизм автоматической отправки уведомлений по электронной почте при выполнении пользовательских условий.

Разработанное программное средство соответствует следующим требованиям:

- Обеспечение централизованного хранения рыночных и финансовых данных компаний.
- Обеспечение автоматического обновления данных из внешнего источника.

- Предоставление пользователю инструменты для анализа стоимости акций и финансовых показателей компаний.
- Обеспечение наглядного отображения информации с использованием графиков.
- Предоставление возможности сравнения нескольких акций по выбранным показателям.
- Обеспечение настройки пользовательских уведомлений по заданным условиям.
- Обеспечение корректной работы механизмов автоматической отправки уведомлений.
- Обеспечение сохранности данных и стабильную работу системы при регулярном обновлении информации.

1.3 Актуальность разработки

Актуальность данной работы обусловлена необходимостью повышения удобства и эффективности анализа акций для частных инвесторов. При принятии инвестиционных решений пользователю требуется учитывать не только текущую стоимость ценных бумаг, но и финансовое состояние компаний, показатели их деятельности и изменение ключевых инвестиционных коэффициентов во времени.

Традиционный подход к проведению фундаментального анализа предполагает использование нескольких информационных ресурсов одновременно. Как правило, данные о котировках акций, финансовой отчетности компаний, дивидендных выплатах и расчетных показателях размещаются на различных платформах. В результате инвестору приходится самостоятельно собирать и сопоставлять информацию из нескольких источников, что увеличивает временные затраты и снижает удобство анализа.

Дополнительной проблемой является необходимость постоянного контроля интересующих показателей. Для отслеживания благоприятных моментов для покупки или анализа изменения финансового состояния компаний пользователь вынужден регулярно проверять значения цены акции,

коэффициентов P/E и P/B, а также других характеристик бизнеса. При одновременном наблюдении за несколькими компаниями такой процесс становится трудоемким и требует значительного количества времени.

Существующие инвестиционные сервисы предоставляют инструменты для отображения рыночной информации и финансовых показателей, однако возможности автоматического уведомления пользователей в большинстве случаев ограничиваются отслеживанием изменения стоимости акций. Контроль фундаментальных показателей компаний зачастую требует самостоятельного мониторинга данных без использования автоматизированных средств оповещения.

В связи с этим актуальной является разработка веб-приложения, объединяющего в рамках единой системы хранение рыночных и финансовых данных, расчет инвестиционных коэффициентов, построение графиков и автоматическое уведомление пользователей о достижении заданных значений цены акции и показателей P/E и P/B.

1.4 Аналоги программы

Для определения требований к разрабатываемой системе был проведен анализ существующих программных решений, предназначенных для получения информации о состоянии фондового рынка и проведения фундаментального анализа компаний. Полностью идентичных решений, объединяющих инструменты визуализации рыночных данных, расчет инвестиционных коэффициентов и систему пользовательских уведомлений по фундаментальным показателям, обнаружено не было. Однако на рынке присутствует ряд сервисов, отдельные функции которых пересекаются с задачами разрабатываемого приложения.

Финам [1] — инвестиционная платформа, предоставляющая доступ к биржевым котировкам, финансовой отчетности компаний и основным инвестиционным показателям. Также реализована возможность получения уведомлений об изменении рыночной цены акций. При этом значения коэффициентов P/E, P/B и P/S рассчитываются на основе опубликованной

финансовой отчетности и обновляются после выхода новых отчетных данных. Интерфейс программы представлен на рисунке 1.

Smart-Lab [2] — информационно-аналитический ресурс, содержащий сведения о котировках акций, дивидендных выплатах, финансовой отчетности и отдельных инвестиционных показателях компаний. Сервис предоставляет пользователям значительный объем аналитической информации, однако набор рассчитываемых показателей ограничен по сравнению с рассматриваемым приложением. Возможности настройки уведомлений по пользовательским условиям отсутствуют. Интерфейс программы представлен на рисунке 2.

Investemika [3] — сервис, ориентированный на фундаментальный анализ российских компаний. Пользователям доступны данные финансовой отчетности за различные периоды, а также расчет инвестиционных коэффициентов. В отличие от большинства рассмотренных решений значения мультипликаторов регулярно пересчитываются с учетом изменения рыночной цены акций. Вместе с тем сервис не предоставляет возможности автоматического уведомления пользователей о достижении заданных условий. Интерфейс программы представлен на рисунке 3.

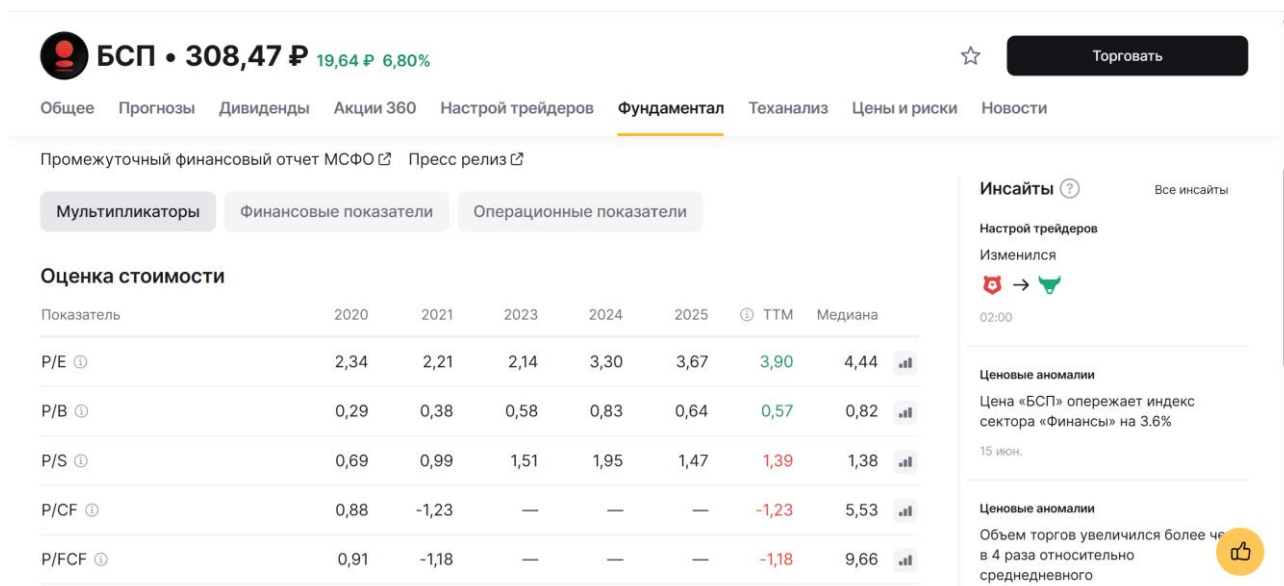
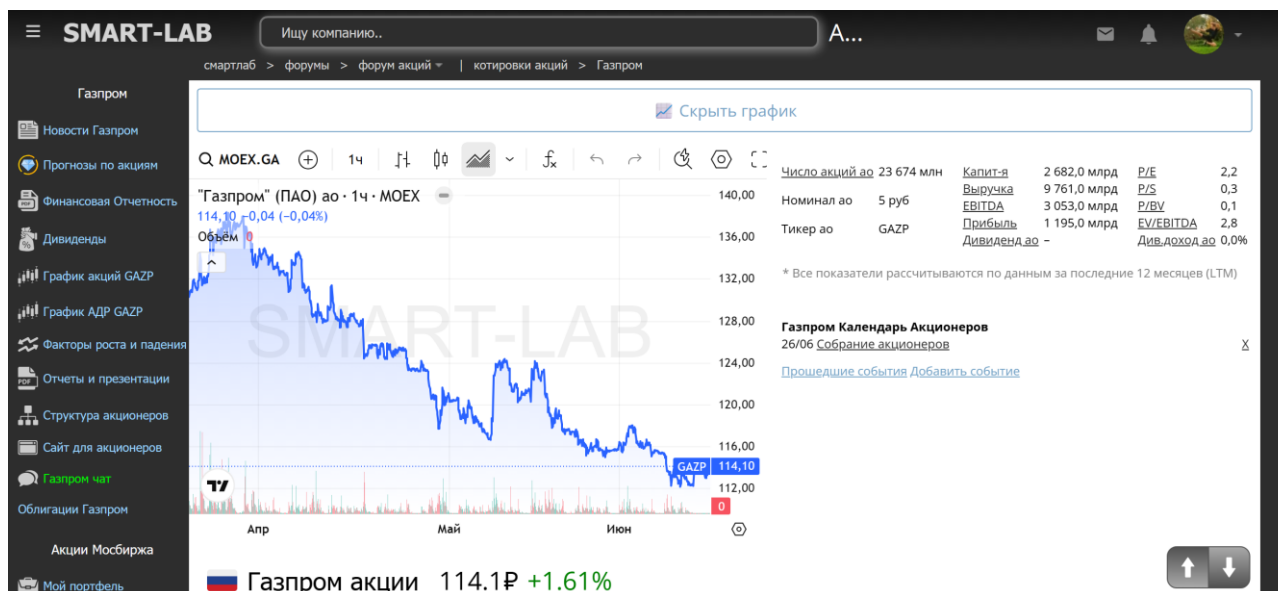


Рисунок 1 – Интерфейс сервиса Финам [1]



investemika

Введите имя компании...

Лукойл (LKOH)

Цена акции: 4 776,5

Капитализация: 3 309 473 312 193 ⓘ

Отчетность и коэффициенты

График

Финансовая отчетность по МСФО (в млн. рублей)

	2025	2024	2023	2021
Выручка	3 767 768	8 621 561	7 928 303	9 435 143
Операционная прибыль	526 646	1 191 817	1 427 697	978 945
Чистая прибыль	-1 059 859	851 546	1 160 271	775 513
Прибыль, относящаяся к акционерам	-1 064 269	848 514	1 154 664	773 442
Активы всего	5 474 421	9 282 619	8 600 173	6 864 749
Текущие активы	1 128 446	3 309 453	2 994 282	2 149 157

2 Решение задачи

2.1 Выбор среды программирования

Для реализации проекта требовалось выбрать среду разработки, обеспечивающую удобную работу с исходным кодом, средствами отладки и инструментами управления проектом. В качестве основных вариантов были рассмотрены следующие программные продукты:

- PyCharm [4] — специализированная интегрированная среда разработки для языка Python. Среда предоставляет развитые средства анализа кода, отладки приложений, работы с базами данных и системами контроля версий. Также поддерживается интеграция с популярными веб-фреймворками, включая Flask и Django.
- Visual Studio Community 2022 [5] — бесплатная интегрированная среда разработки компании Microsoft. Поддерживает работу с различными языками программирования, включая Python, предоставляет инструменты отладки, навигации по коду, управления проектами и интеграцию с системами контроля версий.

По итогам анализа выбор был сделан в пользу Visual Studio Community 2022. Данная среда разработки обладает всеми необходимыми средствами для создания веб-приложения на языке Python, обеспечивает удобную работу с проектом, поддержку отладки и навигации по исходному коду. Кроме того, Visual Studio Community предоставляет единое рабочее пространство для разработки, тестирования и сопровождения программного обеспечения, что позволило эффективно использовать её на всех этапах выполнения работы.

2.2 Выбор языка программирования

Ключевым требованием к языку программирования являлась возможность быстрой разработки веб-приложения, наличие библиотек для взаимодействия с базами данных и внешними источниками информации, а также поддержка инструментов автоматизации и обработки данных. В качестве основных вариантов были рассмотрены следующие языки программирования:

- Python [6] — высокоуровневый язык программирования общего назначения, обладающий простым синтаксисом и развитой экосистемой библиотек. Python широко применяется для веб-разработки, анализа данных, автоматизации процессов и создания программных средств, связанных с обработкой информации. Для языка доступны готовые библиотеки для взаимодействия с базами данных, получения данных из внешних источников и построения веб-приложений.
- C++ [7] — компилируемый язык программирования, отличающийся высокой производительностью и широкими возможностями управления системными ресурсами. Данный язык активно используется при разработке высоконагруженных и ресурсоемких приложений, однако создание веб-приложений на его основе требует большего объема программного кода и использования дополнительных программных средств.

По результатам анализа был выбран язык Python. Основным фактором выбора стало наличие готовых библиотек и инструментов, необходимых для реализации поставленных задач. В частности, библиотека Flask используется для разработки веб-приложения, psycopg2 обеспечивает взаимодействие с базой данных PostgreSQL, requests применяется для получения данных из внешних источников, а smtplib позволяет реализовать механизм отправки уведомлений по электронной почте. Использование Python позволило сократить время разработки и сосредоточиться на реализации функциональных возможностей системы.

2.3 Выбор СУБД

Для хранения информации о котировках акций, финансовой отчетности компаний, пользовательских подписках и истории изменения показателей необходимо было выбрать систему управления базами данных. При выборе учитывались возможности хранения больших объемов исторических данных, поддержка сложных запросов и удобство интеграции с языком программирования Python.

В качестве основных вариантов были рассмотрены следующие системы управления базами данных:

- PostgreSQL [8] — объектно-реляционная система управления базами данных с открытым исходным кодом. Поддерживает широкий набор типов данных, включая JSONB, транзакции, индексы и сложные SQL-запросы. PostgreSQL широко используется при разработке веб-приложений и аналитических систем.
- MySQL [9] — популярная реляционная система управления базами данных, широко применяемая при разработке веб-приложений. Поддерживает хранение структурированных данных, обладает высокой производительностью и развитым набором средств администрирования.

По результатам анализа была выбрана СУБД PostgreSQL. Основным фактором выбора стала поддержка типа данных JSONB, который используется в разработанном приложении для хранения пользовательских условий подписки. Кроме того, имеется практический опыт работы с PostgreSQL, что позволило сократить время проектирования и реализации базы данных.

2.4 Выбор системы построения интерфейса

Для разрабатываемого веб-приложения требовалось создать пользовательский интерфейс, обеспечивающий удобный просмотр информации о котировках акций, финансовых показателях компаний и настройку пользовательских уведомлений. При выборе средств построения интерфейса учитывались удобство разработки, гибкость настройки внешнего вида и возможность интеграции с серверной частью приложения.

В качестве основных вариантов были рассмотрены следующие решения:

- Django Templates [10] — встроенная система шаблонов веб-фреймворка Django, позволяющая формировать HTML-страницы на стороне сервера. Данный подход обеспечивает широкие возможности по созданию пользовательского интерфейса, однако предполагает использование самого фреймворка Django и его архитектурных компонентов.

- Streamlit [11] — библиотека для быстрого создания веб-интерфейсов на языке Python. Позволяет создавать формы, таблицы и графики без написания HTML и CSS. Основным преимуществом является высокая скорость разработки, однако возможности настройки интерфейса и структуры веб-приложения ограничены по сравнению с классическими веб-фреймворками.
- Flask с использованием шаблонизатора Jinja2 [12] — легковесный веб-фреймворк, позволяющий формировать HTML-страницы на стороне сервера и обеспечивающий гибкую организацию структуры приложения. Использование шаблонов Jinja2 позволяет динамически отображать данные, полученные из базы данных, а также интегрировать HTML, CSS и JavaScript для создания пользовательского интерфейса.

По результатам анализа был выбран вариант Flask с использованием шаблонизатора Jinja2. Данное решение обеспечивает достаточную гибкость при разработке пользовательского интерфейса, позволяет реализовать интерактивные графики, таблицы и формы настройки уведомлений, а также легко интегрируется с серверной логикой приложения и базой данных PostgreSQL. Использование Flask позволило реализовать необходимый функционал без избыточной сложности архитектуры и сохранить возможность дальнейшего расширения системы.

2.5 Проектирование базы данных

Ядром разработанного веб-приложения является реляционная база данных PostgreSQL. База данных предназначена для хранения информации об акциях российских компаний, исторических данных о котировках, финансовой отчетности, дивидендных выплатах, количестве акций в обращении, а также пользовательских подписках на уведомления. На рисунке 4 представлена схема разработанной базы данных.

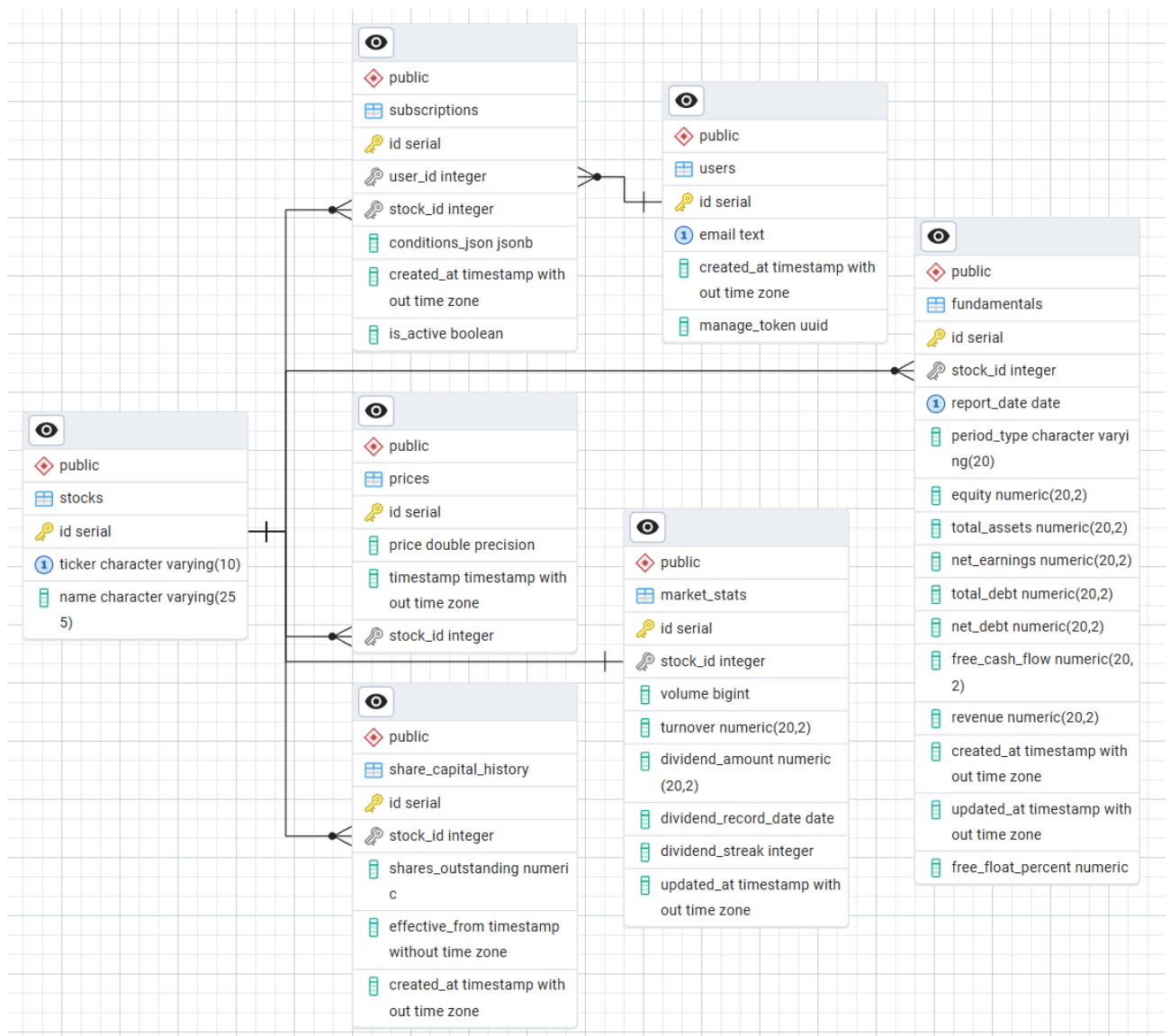


Рисунок 4 – Схема базы данных

Описание таблиц и полей:

1. stocks (Акции)

- id — уникальный идентификатор акции.
- ticker — биржевой тикер компании.
- name — наименование компании.

2. prices (Исторические котировки)

- id — уникальный идентификатор записи.
- stock_id — ссылка на акцию из таблицы stocks.
- price — стоимость акции.
- timestamp — дата и время фиксации стоимости.

3. fundamentals (Финансовая отчетность)

- id — уникальный идентификатор записи.
- stock_id — ссылка на акцию.
- report_date — дата финансовой отчетности.
- period_type — тип отчетного периода.
- equity — собственный капитал компании.
- total_assets — общая стоимость активов.
- net_earnings — чистая прибыль.
- total_debt — общий долг компании.
- net_debt — чистый долг.
- free_cash_flow — свободный денежный поток.
- revenue — выручка компании.
- free_float_percent — доля акций в свободном обращении.

4. market_stats (Рыночные показатели)

- id — уникальный идентификатор записи.
- stock_id — ссылка на акцию.
- volume — объем торгов за торговую сессию.
- turnover — денежный оборот торгов.
- dividend_amount — размер последней дивидендной выплаты.
- dividend_record_date — дата закрытия реестра для получения дивидендов.
- dividend_streak — количество последовательных лет выплаты дивидендов.

5. share_capital_history (История количества акций в обращении)

- id — уникальный идентификатор записи.
- stock_id — ссылка на акцию.
- shares_outstanding — количество выпущенных акций компании.
- effective_from — дата начала действия значения.

6. users (Пользователи)

- id — уникальный идентификатор пользователя.
- email — адрес электронной почты пользователя.

- `created_at` — дата регистрации пользователя.
- `manage_token` — уникальный идентификатор для управления подписками.

7. subscriptions (Подписки на уведомления)

- `id` — уникальный идентификатор подписки.
- `user_id` — ссылка на пользователя.
- `stock_id` — ссылка на акцию.
- `conditions_json` — набор пользовательских условий для отправки уведомлений.
- `created_at` — дата создания подписки.
- `is_active` — статус подписки.

Особенностью таблицы является использование типа данных JSONB для хранения пользовательских условий подписки. Такой подход позволяет хранить произвольное количество условий без изменения структуры базы данных.

Связи между сущностями:

Акция ↔ Исторические котировки (1:N): одна акция может иметь множество записей о стоимости за различные моменты времени.

Акция ↔ Финансовая отчетность (1:N): для одной компании может храниться несколько финансовых отчетов за различные периоды.

Акция ↔ Рыночные показатели (1:1): каждой акции соответствует одна актуальная запись рыночной статистики.

Акция ↔ История количества акций (1:N): количество акций в обращении может изменяться с течением времени.

Пользователь ↔ Подписка (1:N): один пользователь может создать несколько подписок на различные акции.

Акция ↔ Подписка (1:N): на одну акцию может быть оформлено множество пользовательских подписок.

Для повышения производительности системы в базе данных используются индексы. Индексы позволяют сократить время выполнения запросов и ускорить выборку данных при работе с историческими котировками и финансовой информацией.

В разработанной системе используются следующие основные индексы:

- `stocks_ticker_key` — обеспечивает быстрый поиск акций по тикеру и исключает возможность создания дублирующих записей.
- `idx_prices_stock_id` — ускоряет выборку исторических котировок для конкретной акции.
- `idx_prices_timestamp` — ускоряет выборку данных за определенный временной период.
- `unique_price_point` — предотвращает сохранение нескольких значений цены одной акции для одинакового момента времени.
- `uq_fundamentals_report` — обеспечивает уникальность финансовой отчетности компании за одну отчетную дату.
- `users_email_key` — обеспечивает уникальность адресов электронной почты пользователей и ускоряет поиск пользователей по адресу электронной почты.

2.6 Разработка программы

2.6.1 Выбранные библиотеки

Для реализации проекта был использован ряд ключевых Python-библиотек, каждая из которых решает свою специфическую задачу.

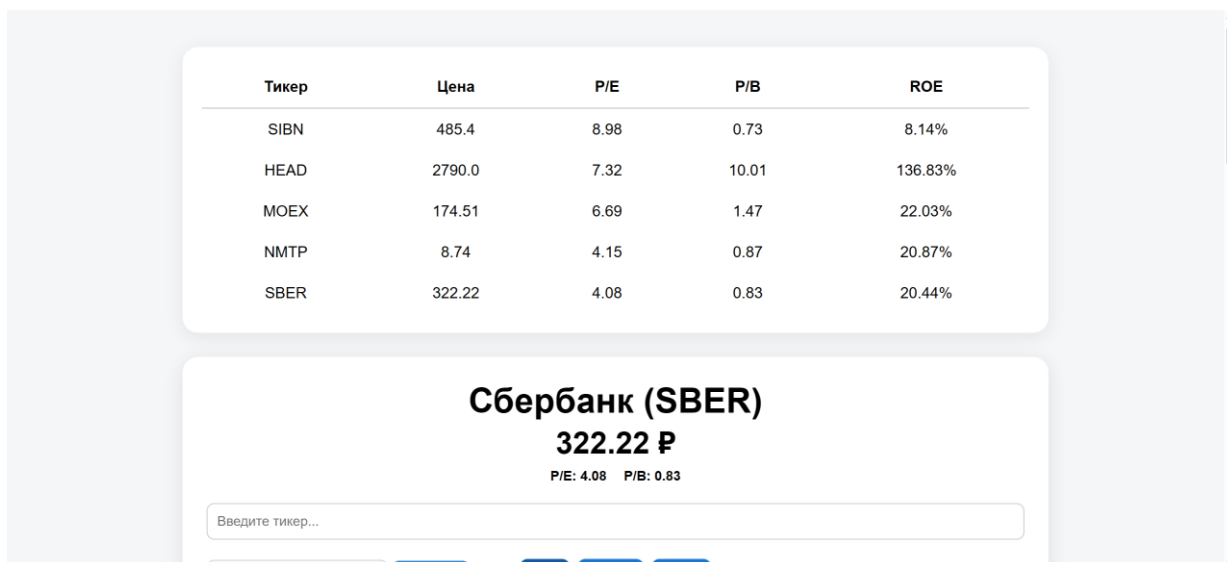
1. Flask [13] — микрофреймворк для разработки веб-приложений на языке Python. В разработанной системе Flask используется для обработки HTTP-запросов пользователей, маршрутизации страниц, формирования ответов сервера и взаимодействия между пользовательским интерфейсом и базой данных. Данная библиотека является основой серверной части приложения.
2. `psycopg2` [14] — библиотека для взаимодействия с системой управления базами данных PostgreSQL. Она обеспечивает выполнение SQL-запросов, получение данных из базы данных и сохранение информации о котировках акций, финансовой отчетности и пользовательских подписках.

3. requests [15] — библиотека для выполнения HTTP-запросов. В проекте используется для получения информации о стоимости акций, финансовой отчетности компаний, дивидендных выплатах и других данных из внешних источников.
4. python-dotenv [16] — библиотека для загрузки параметров конфигурации из файлов окружения. Используется для хранения настроек подключения к базе данных и параметров почтового сервера без размещения конфиденциальной информации непосредственно в исходном коде программы.
5. smtplib [17] — стандартная библиотека Python для отправки электронной почты по протоколу SMTP. В разработанном приложении используется для формирования и отправки уведомлений пользователям при достижении заданных условий по стоимости акций и инвестиционным коэффициентам.
6. email.mime [18] — набор модулей для формирования электронных писем. Используется совместно с библиотекой smtplib для создания текстовых сообщений и формирования структуры электронных уведомлений, отправляемых пользователям.

2.6.2 Итоговая версия программы

В результате выполнения выпускной квалификационной работы было разработано веб-приложение для мониторинга акций российских компаний, акции которых обращаются на Московской бирже. Приложение предназначено для получения актуальной информации о состоянии фондового рынка, анализа финансовых показателей компаний и автоматизации процесса отслеживания инвестиционных событий с помощью системы уведомлений.

После запуска приложения пользователю отображается основной интерфейс системы (Рисунок 5).



The screenshot displays a web application interface. At the top, there is a table with five columns: 'Тикер' (Ticker), 'Цена' (Price), 'P/E', 'P/B', and 'ROE'. Below this table, there is a detailed view for 'Сбербанк (SBER)' showing its current price '322.22 Р' and its P/E and P/B ratios. At the bottom, there is a search bar with the placeholder text 'Введите тикер...'.

Тикер	Цена	P/E	P/B	ROE
SIBN	485.4	8.98	0.73	8.14%
HEAD	2790.0	7.32	10.01	136.83%
MOEX	174.51	6.69	1.47	22.03%
NMTP	8.74	4.15	0.87	20.87%
SBER	322.22	4.08	0.83	20.44%

Сбербанк (SBER)
322.22 Р
P/E: 4.08 P/B: 0.83

Введите тикер...

Рисунок 5 – Основной интерфейс разработанного веб-приложения

В верхней части страницы расположена сводная таблица акций, предназначенная для быстрого сравнения компаний между собой. Таблица отображает пять акций и содержит следующие столбцы:

- тикер компании;
- текущая цена акции;
- коэффициент P/E;
- коэффициент P/B;
- коэффициент ROE.

Логика реализации:

Для каждого столбца реализована возможность сортировки по возрастанию и убыванию. Это позволяет пользователю быстро находить компании с наиболее привлекательными или, наоборот, наименее привлекательными значениями инвестиционных показателей.

Для формирования сводной таблицы акций приложение выполняет SQL-запрос к базе данных PostgreSQL, объединяя данные сразу из нескольких таблиц: stocks, prices, fundamentals и share_capital_history. Такой подход позволяет получить все необходимые показатели одним запросом без выполнения дополнительных обращений к базе данных.

В первую очередь для каждой компании определяется актуальная рыночная цена акции. Для этого из таблицы `prices` выбирается последняя запись по времени для каждого тикера. Аналогичным образом из таблицы `fundamentals` извлекаются последние доступные значения чистой прибыли и собственного капитала компании, а из таблицы `share_capital_history` — актуальное количество размещённых акций.

После получения исходных данных непосредственно в SQL-запросе рассчитываются основные инвестиционные коэффициенты:

- ROE (Return on Equity) — отношение чистой прибыли к собственному капиталу компании;
- P/E (Price to Earnings) — отношение рыночной капитализации компании к её чистой прибыли;
- P/B (Price to Book) — отношение рыночной капитализации компании к величине собственного капитала.

Для предотвращения ошибок вычисления выполняется дополнительная проверка значений на отсутствие данных и деление на ноль. Если расчёт показателя невозможен, пользователю отображается пустое значение.

После выполнения запроса полученные данные преобразуются в список словарей Python, содержащий тикер компании, текущую цену акции и рассчитанные коэффициенты (Листинг 1).

```
cur.execute("""
select
    s.ticker,
    p.price,
    case
        when f.net_earnings is not null
            and f.equity is not null
            and f.equity != 0
        then round(
            (
```

```

        f.net_earnings
        / f.equity
        * 100
    )::numeric,
    2
)
else null
end as roe,
case
    when f.net_earnings is not null
        and f.net_earnings != 0
        and sch.shares_outstanding is not null
    then round(
        (
            (
                p.price * sch.shares_outstanding
            ) / f.net_earnings
        )::numeric,
        2
    )
    else null
end as pe,
case
    when f.equity is not null
        and f.equity != 0
        and sch.shares_outstanding is not null
    then round(
        (
            (
                p.price * sch.shares_outstanding

```

```

        ) / f.equity
    )::numeric,
    2
)
else null
end as pb
from stocks s
join (
    select distinct on (stock_id)
        stock_id,
        price
    from prices
    order by stock_id, timestamp desc
) p
on p.stock_id = s.id
left join (
    select distinct on (stock_id)
        stock_id,
        net_earnings,
        equity
    from fundamentals
    order by stock_id, report_date desc
) f
on f.stock_id = s.id
left join (
    select distinct on (stock_id)
        stock_id,
        shares_outstanding
    from share_capital_history
    order by stock_id, effective_from desc

```

```

) sch
    on sch.stock_id = s.id
    order by pe desc
""")
market_table = cur.fetchall()
stocks_table = []
for row in market_table:
    stocks_table.append({
        "ticker": row[0],
        "price": row[1],
        "roe": row[2],
        "pe": row[3],
        "pb": row[4]
    })

```

Листинг 1 – Формирование сводной таблицы акций и расчёт инвестиционных коэффициентов

Под таблицей отображается краткая информация о выбранной компании. В центральной части страницы выводится тикер акции, её текущая рыночная цена, а также актуальные значения коэффициентов P/E и P/B.

Данный блок предназначен для быстрого ознакомления пользователя с основными характеристиками выбранной компании без необходимости изучения дополнительных разделов страницы.

Логика реализации:

После выбора пользователем акции приложение получает её тикер из параметров HTTP-запроса. Если тикер не указан, по умолчанию отображается акция SBER. Далее выполняется поиск выбранной акции в таблице stocks и получение её идентификатора.

Для отображения текущей информации на странице система получает последнюю доступную цену акции из таблицы prices, а также актуальные данные

о чистой прибыли, собственном капитале и количестве акций в обращении. На основе этих данных рассчитываются коэффициенты P/E, P/B, P/S. Полученные значения передаются в HTML-шаблон и отображаются под названием выбранной акции (Листинг 2).

```
ticker = request.args.get(
    "ticker",
    "SBER"
).upper()
...
cur.execute("""
    select
        id,
        ticker,
        name
    from stocks
    where ticker = %s
""", (ticker,))

stock = cur.fetchone()
stock_id = stock[0]
...
cur.execute("""
    select max(timestamp)
    from prices
    where stock_id = %s
""", (stock_id,))
latest_timestamp = cur.fetchone()[0]
...
cur.execute("""
    select price
```

```

from prices
where stock_id = %s
order by timestamp desc
limit 1
"""', (stock_id,))
last_price = cur.fetchone()[0]
...
market_cap = (
    last_price * float(shares_outstanding)
    if last_price and shares_outstanding
    else None
)
p_e = (
    round(
        market_cap / float(net_earnings),
        2
    )
    if market_cap
    and net_earnings
    and float(net_earnings) != 0
    else "—"
)
p_b = (
    round(
        market_cap / float(equity),
        2
    )
    if market_cap
    and equity
    and float(equity) != 0

```

```

        else "—"
    )
    ...
    return render_template(
        "index.html",
        ticker=ticker,
        last_price=last_price,
        p_e=p_e,
        p_b=p_b,
        ...
    )

```

Листинг 2 – Получение информации о выбранной акции

Ниже располагается блок выбора компании. Пользователь может выбрать интересующий тикер из выпадающего списка. После выбора данные на странице автоматически обновляются.

Для анализа исторических данных предусмотрен набор элементов управления отображением графика. Пользователь может выбрать один из временных интервалов:

- день;
- неделя;
- месяц.

Дополнительно реализована возможность переключения между отображаемыми показателями:

- цена акции;
- коэффициент P/E;
- коэффициент P/B.

Такой подход позволяет анализировать как динамику стоимости акции, так и изменение фундаментальных показателей компании во времени.

В левой части данного блока реализован механизм сравнения нескольких компаний. Пользователь может выбрать дополнительную акцию из списка и добавить её на график. После добавления тикер отображается в отдельном списке ниже. Для удаления компании из сравнения предусмотрена кнопка удаления рядом с тикером (Рисунок 6).



Рисунок 6 – Блок управления отображением графика и сравнения компаний

Логика реализации:

Пользователю предоставляется возможность изменять отображаемые на графике данные без перезагрузки приложения. Для этого используются параметры URL-запроса, которые передаются в серверную часть при выборе временного интервала или финансового показателя. В зависимости от выбранных значений система автоматически получает необходимые данные из базы данных и формирует новый набор данных для построения графика.

Для отображения доступны три временных интервала: день, неделя и месяц. Также пользователь может переключаться между графиками цены акции, коэффициента Р/Е и коэффициента Р/В. Выбранные параметры считываются из строки запроса и используются при дальнейшем формировании данных для визуализации (Листинг 3).

```
period = request.args.get(
    "period",
    "day"
)
metric = request.args.get(
    "metric",
    "price"
).lower()
...
interval = {
    "day": "1 day",
    "week": "7 days",
    "month": "30 days"
}.get(period, "1 day")
...
if metric == "price":
    values = current_prices
elif metric == "pe":
    values = current_pe
else:
    values = current_pb
```

Листинг 3 – Переключение отображаемого показателя

Для сравнения нескольких акций пользователь может добавлять дополнительные тикеры. Выбранные компании передаются через параметры

URL и объединяются в единый список. Далее для каждой компании отдельно выполняется получение исторических данных и формируется собственный набор значений, который отображается на общем графике отдельной линией (Листинг 4).

```
compare = request.args.get(
    "compare",
    ""
)
compare_tickers = [
    t.strip().upper()
    for t in compare.split(",")
    if t.strip()
]
all_tickers = [ticker]
for t in compare_tickers:
    if t not in all_tickers:
        all_tickers.append(t)
...
for dataset_index, current_ticker in enumerate(all_tickers):
    ...
    chart_datasets.append({
        "label": current_ticker,
        "data": values,
        "borderColor": colors[
            dataset_index % len(colors)
        ],
        "borderWidth": 2,
        "pointRadius": 0,
        "pointHoverRadius": 4,
        "fill": False,
```

```
"tension": 0  
})
```

Листинг 4 – Добавление и удаление компаний для сравнения

Основную часть интерфейса занимает интерактивный график, отображающий выбранный пользователем показатель. В зависимости от настроек график показывает изменение рыночной цены акции либо динамику коэффициентов P/E и P/B.

Для удобства анализа под графиком расположен ползунок диапазона отображения, позволяющий изменять рассматриваемый временной интервал без повторного выбора периода (Рисунок 7).

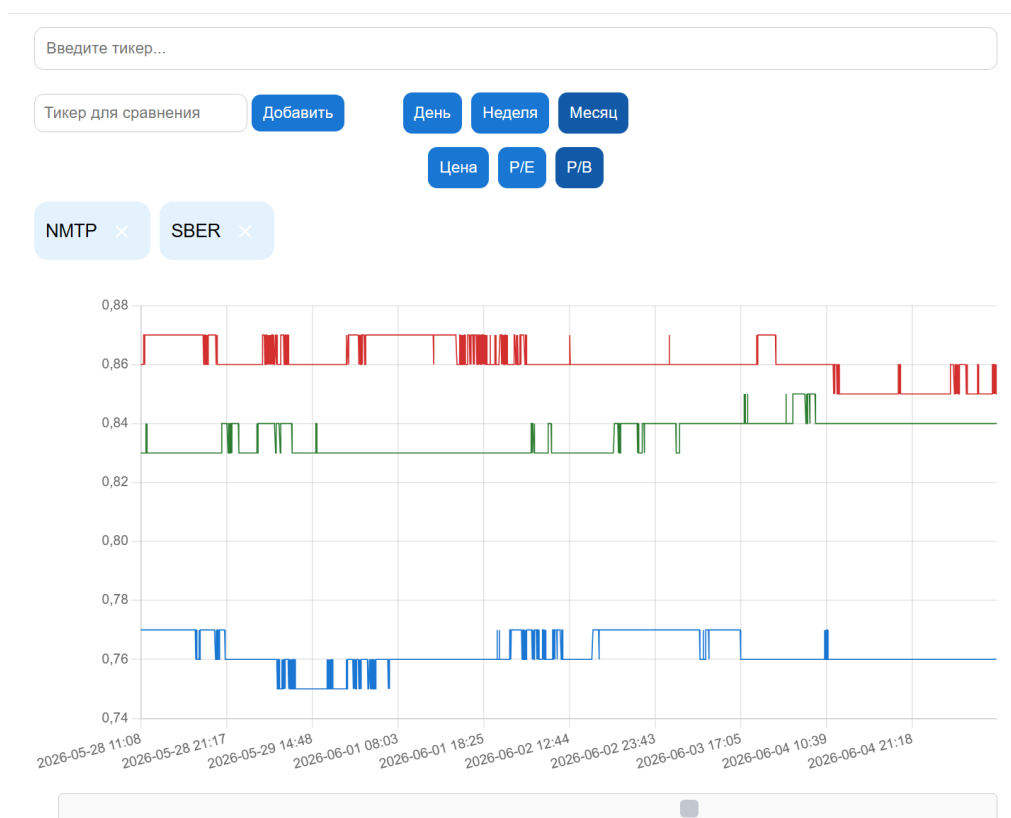


Рисунок 7 – Интерфейс отображения исторических данных акции

Логика реализации:

Для построения графика приложение получает исторические данные по выбранной акции из таблицы `prices`. В зависимости от выбранного пользователем временного интервала формируется SQL-запрос, который

извлекает значения цены и времени их фиксации за необходимый период. Полученные данные преобразуются в массивы, которые впоследствии передаются в шаблон страницы и используются библиотекой визуализации для построения графика (Листинг 5).

Дополнительно реализована возможность сравнения нескольких компаний на одном графике. Для каждой выбранной акции выполняется отдельное получение исторических данных, после чего формируется собственный набор значений. Все сформированные наборы объединяются в коллекцию `chart_datasets`, которая затем используется для отображения нескольких линий на одном графике и их последующего сравнения пользователем (Листинг 5).

```
cur.execute("""
    select
        price,
        timestamp
    from prices
    where stock_id = %s
    and timestamp >= %s::timestamp - interval %s
    order by timestamp
""", (
    stock_id,
    latest_timestamp,
    interval
))
data = cur.fetchall()
...
timestamps = [
    x[1].strftime("%Y-%m-%d %H:%M")
    for x in data
]
...
```

```

for dataset_index, current_ticker in enumerate(all_tickers):
    ...
    cur.execute("""
        select
            price,
            timestamp
        from prices
        where stock_id = %s
        and timestamp >= %s::timestamp - interval %s
        order by timestamp
    """, (
        current_stock_id,
        current_latest_timestamp,
        interval
    ))
    current_data = cur.fetchall()
    ...
    chart_datasets.append({
        "label": current_ticker,
        "data": values,
        "borderColor": colors[
            dataset_index % len(colors)
        ],
        "borderWidth": 2,
        "pointRadius": 0,
        "pointHoverRadius": 4,
        "fill": False,
        "tension": 0
    })

```

Листинг 5 – Подготовка данных для построения графика

Под графиком расположен блок фундаментальных показателей компании. Показатели отображаются в виде информационных карточек по два элемента в строке.

В данном разделе представлены сведения, полученные из финансовой отчетности компании:

- собственный капитал;
- активы;
- чистая прибыль;
- выручка;
- общий долг;
- чистый долг;
- свободный денежный поток;
- free-float и другие показатели.

Данный блок позволяет пользователю получить информацию, необходимую для проведения фундаментального анализа компании (Рисунок 8).

P/E 1.9	P/B 0.13
ROE 6.92 %	P/S 0.25
FCF/E -0.74	Net Debt/E 4.67
Equity (B) 18 894 млрд ₽	Total Assets 31 891 млрд ₽
Revenue (S) 9 771 млрд ₽	Net Earnings (E) 1 307 млрд ₽
Total Debt 6 738 млрд ₽	Net Debt 6 104 млрд ₽
Free Cash Flow (FCF) -961 млрд ₽	Free Float 47 %
Turnover 7 344 908 670	Volume 69 242 740 шт.
Shares Outstanding 23 673 512 900 шт.	Liquidity 53 177 шт/с

Рисунок 8 – Блок фундаментальных показателей компании

Логика реализации:

Для отображения фундаментальных показателей приложения используются данные из таблиц fundamentals, market_stats и share_capital_history. Из таблицы fundamentals извлекаются значения собственного капитала, чистой прибыли, выручки, свободного денежного потока и других показателей финансовой отчётности компании. Из таблицы share_capital_history загружается актуальное количество размещённых акций, необходимое для расчёта рыночной капитализации и инвестиционных коэффициентов.

После получения данных выполняется вычисление основных мультипликаторов, используемых инвесторами при анализе акций. Для расчётов сначала определяется рыночная капитализация компании как произведение текущей цены акции на количество размещённых акций. Затем на её основе вычисляются коэффициенты P/E и P/B. Дополнительно рассчитывается показатель рентабельности собственного капитала ROE, который позволяет оценить эффективность использования капитала компании (Листинг 6, Листинг 7, Листинг 8).

```
market_cap = (  
    last_price * float(shares_outstanding)  
    if last_price and shares_outstanding  
    else None)  
p_e = (  
    round(  
        market_cap / float(net_earnings),  
        2)  
    if market_cap  
    and net_earnings  
    and float(net_earnings) != 0  
    else "—")
```

Листинг 6 – Расчёт коэффициента P/E

```

market_cap = (
    last_price * float(shares_outstanding)
    if last_price and shares_outstanding
    else None)
p_b = (
    round(
        market_cap / float(equity),
        2)
    if market_cap
    and equity
    and float(equity) != 0
    else "—")

```

Листинг 7 – Расчёт коэффициента P/B

```

roe = (
    round(
        float(net_earnings)
        / float(equity)
        * 100,
        2)
    if net_earnings
    and equity
    and float(equity) != 0
    else "—")

```

Листинг 8 – Расчёт коэффициента ROE

Ниже располагается блок дивидендной информации. Пользователю отображаются сведения о последних дивидендных выплатах компании, размере дивиденда и дате закрытия реестра акционеров.

Данный раздел предназначен для инвесторов, ориентированных на получение дивидендного дохода (Рисунок 9).

Dividend Amount	Dividend Yield
51.03 ₽	45.42%
Dividend Record Date	Dividend Streak
11.10.2022	10 лет

Рисунок 9 – Блок отображения дивидендной информации

Логика реализации:

Для отображения информации о дивидендах приложение обращается к таблице `market_stats`, в которой хранятся сведения о размере последней выплаты, дате закрытия реестра акционеров и продолжительности дивидендной истории компании. После получения данных выполняется дополнительная обработка: рассчитывается дивидендная доходность как отношение размера дивиденда к текущей цене акции, а также производится форматирование значений для последующего отображения в интерфейсе пользователя (Листинг 9).

```
cur.execute("""
select
    dividend_amount,
    dividend_record_date,
    dividend_streak
from market_stats
where stock_id = %s
""", (stock_id,))
dividend_data = cur.fetchone()
...
if market_row:
    dividend_amount = market_row[2]
    dividend_record_date = market_row[3]
```

```

        dividend_streak = market_row[4]
else:
    dividend_amount = None
    dividend_record_date = None
    dividend_streak = None
...
if dividend_amount and prices:
    dividend_yield = round(
        (
            float(dividend_amount)
            / float(prices[-1])
        ) * 100,
        2
    )
else:
    dividend_yield = None

```

Листинг 9 – Получение информации о дивидендах

В нижней части страницы располагается система подписок на уведомления. Для создания подписки пользователь указывает адрес электронной почты, на который будут отправляться уведомления. После ввода адреса пользователь может сформировать одно или несколько условий срабатывания уведомления. Каждое условие состоит из контролируемого показателя и его порогового значения. Для повышения гибкости настройки предусмотрена возможность добавления дополнительных условий с помощью кнопки «Добавить условие». Пользователь может создать до трёх независимых условий в рамках одной подписки (Рисунок 10). После нажатия кнопки «Подписаться» информация сохраняется в базе данных и начинает участвовать в автоматической проверке.

Рисунок 10 – Интерфейс создания подписки на уведомления

Логика реализации:

После получения данных формы приложение формирует список условий в формате JSON. Такой подход позволяет хранить произвольное количество условий для одной подписки без изменения структуры базы данных. Полученный JSON-объект впоследствии сохраняется в поле `conditions_json` таблицы `subscriptions`, имеющее тип `JSONB` (Листинг 10).

```

conditions = request.form.getlist(
    "condition[]"
)
values = request.form.getlist(
    "value[]"
)
conditions_json = []
for condition, value in zip(
    conditions,
    values
):
    conditions_json.append({
        "condition": condition,

```

```

        "value": float(value)
    })
conditions_json = json.dumps(
    conditions_json
)

```

Листинг 10 – Формирование объекта подписки

Перед сохранением подписки выполняется проверка наличия пользователя в таблице users. Если пользователь с указанным адресом электронной почты ещё не существует, создаётся новая запись. Далее определяется идентификатор выбранной акции, после чего данные подписки сохраняются в таблицу subscriptions для последующей обработки сервисом уведомлений (Листинг 11).

```

cursor.execute("""
    insert into users(email)
    values(%s)
    on conflict(email) do nothing
""", (email,))
cursor.execute(
    "select id from users where email = %s",
    (email,)
)
user_id = cursor.fetchone()[0]
...
cursor.execute(
    "select id from stocks where ticker = %s",
    (ticker,)
)
stock_id = cursor.fetchone()[0]
...
cursor.execute("""

```

```

insert into subscriptions
(
    user_id,
    stock_id,
    conditions_json
)
values (%s, %s, %s)
""", (
    user_id,
    stock_id,
    conditions_json
))
conn.commit()

```

Листинг 11 – Сохранение подписки в базу данных

После создания подписки система автоматически проверяет выполнение заданных пользователем условий. При достижении требуемых значений формируется уведомление, которое отправляется на указанный адрес электронной почты.

Данный механизм позволяет пользователю не отслеживать изменения показателей вручную и своевременно получать информацию о важных событиях.

После создания подписки информация о ней сохраняется в таблице subscriptions. Специальный сервис уведомлений периодически выполняет проверку всех активных подписок пользователей. Для каждой подписки из базы данных извлекаются текущие значения цены акции и инвестиционных коэффициентов P/E и P/B. Далее система последовательно проверяет выполнение всех условий, указанных пользователем при создании подписки. Проверка выполняется для каждого условия отдельно. Если хотя бы одно из условий не выполняется, обработка подписки прекращается. Уведомление

отправляется только в случае одновременного выполнения всех заданных условий (Листинг 12).

```
def check_and_notify(
    stock_id,
    ticker,
    price,
    pe,
    pb
):
    subscriptions = get_subscriptions(stock_id)
    for (
        sub_id,
        user_id,
        conditions_json,
        email
    ) in subscriptions:
        conditions = conditions_json
        all_conditions_met = True
        for condition_data in conditions:
            condition = (
                condition_data["condition"]
            )
            target = float(
                condition_data["value"]
            )
            if condition == "price_above":
                if price <= target:
                    all_conditions_met = False
                    break
            elif condition == "price_below":
```

```

    if price >= target:
        all_conditions_met = False
        break
    elif condition == "pe_above":
        if pe is None or pe <= target:
            all_conditions_met = False
            break
    elif condition == "pe_below":
        if pe is None or pe >= target:
            all_conditions_met = False
            break
    elif condition == "pb_above":
        if pb is None or pb <= target:
            all_conditions_met = False
            break
    elif condition == "pb_below":
        if pb is None or pb >= target:
            all_conditions_met = False
            break
    if not all_conditions_met:
        continue

```

Листинг 12 – Проверка условий подписки

Для отправки уведомлений используется SMTP-сервер Gmail. Сообщение формируется в формате HTML и содержит актуальные значения цены акции, коэффициентов, а также список условий, которые привели к срабатыванию уведомления. Дополнительно в письмо включается ссылка на страницу управления подписками пользователя, позволяющая просматривать и удалять существующие подписки. После успешной отправки уведомления подписка

автоматически меняет статус в базе данных на “неактивна”, что предотвращает повторную отправку одинаковых сообщений (Листинг 13).

```
cursor.execute("""
    select manage_token
    from users
    where id = %s
""", (user_id,))
manage_token = cursor.fetchone()[0]
manage_link = (
    f"http://127.0.0.1:5000/manage"
    f"?token={manage_token}"
)
send_email(
    email,
    ticker,
    price,
    pe,
    pb,
    ps,
    fcf_e,
    net_debt_e,
    liquidity,
    dividend_yield,
    dividend_streak,
    conditions,
    manage_link
)

cursor.execute("""
    update subscriptions
```

```
set is_active = false
where id = %s
''', (sub_id,))
```

Листинг 13 – Отправка уведомления пользователю

2.6.3 Дополнительные функции

Помимо основного пользовательского интерфейса для анализа акций и создания подписок, в системе реализован ряд дополнительных функций, обеспечивающих актуальность данных и удобство управления уведомлениями.

Одной из ключевых функций системы является автоматическое обновление информации о котировках и финансовых показателях компаний. Для решения данной задачи был разработан отдельный программный модуль, выполняющий сбор данных с внешних источников и последующее сохранение информации в базу данных PostgreSQL.

Модуль сбора данных работает независимо от веб-приложения и запускается по расписанию. В процессе работы программа получает информацию о текущих котировках акций, финансовой отчетности компаний, количестве акций в обращении, дивидендных выплатах и других показателях, используемых при расчете инвестиционных коэффициентов.

Полученные данные проходят обработку и сохраняются в соответствующие таблицы базы данных. Благодаря этому пользователь всегда работает с актуальной информацией без необходимости вручную обновлять данные или выполнять дополнительные действия.

Логика реализации:

Актуальность отображаемой информации в системе обеспечивается отдельным программным модулем — парсером данных. Его задача заключается в регулярном получении биржевой информации с официального API Московской биржи, обработке полученных данных и сохранении результатов в базу данных приложения. В процессе работы парсер получает текущие котировки акций, информацию о торговых оборотах, объемах торгов,

дивидендах и количестве выпущенных акций, после чего обновляет соответствующие таблицы базы данных (Листинг 14).

```
def fetch_all():
    stocks_map = load_stocks()
    r = requests.get(url)
    r.raise_for_status()
    data = r.json()
    columns = data["marketdata"]["columns"]
    rows = data["marketdata"]["data"]
    idx_sec = columns.index("SECID")
    idx_last = columns.index("LAST")
    result = []
    for row in rows:
        ticker = row[idx_sec].upper()
        price = row[idx_last]
        if ticker not in stocks_map:
            continue
        if price is None:
            continue
        result.append((
            ticker,
            float(price)
        ))
    return result
```

Листинг 14 – Получение данных о котировках акций

После получения данных парсер сохраняет новые значения котировок в таблицу prices. Для каждой акции определяется внутренний идентификатор компании, после чего в таблицу заносится текущая цена и время получения информации. При сохранении используется ограничение уникальности по

комбинации идентификатора акции и временной метки, что исключает появление дублирующихся записей. После записи данных дополнительно выполняется расчёт инвестиционных коэффициентов и проверка пользовательских подписок на уведомления (Листинг 15).

```
def save_data(items):
    now = datetime.now().replace(
        second=0,
        microsecond=0
    )
    stocks_map = load_stocks()
    for ticker, price in items:
        if ticker not in stocks_map:
            continue
        stock_id = stocks_map[ticker]
        cursor.execute("""
            insert into prices (
                stock_id,
                price,
                timestamp
            )
            values (%s, %s, %s)
            on conflict (
                stock_id,
                timestamp
            )
            do nothing
        """, (
            stock_id,
            price,
            now
```

```

))
...
check_and_notify(
    stock_id,
    ticker,
    price,
    pe,
    pb
)
conn.commit()

```

Листинг 15 – Сохранение полученных данных в базу данных

Второй дополнительной функцией системы является механизм управления подписками через персональную ссылку (Magic Link).

После создания подписки пользователю не требуется проходить регистрацию или выполнять вход в систему. Для каждого адреса электронной почты автоматически создается уникальный токен, который используется для формирования персональной ссылки управления подписками.

После перехода по такой ссылке пользователь получает доступ к специальной странице управления подписками, где отображаются все активные уведомления, привязанные к его адресу электронной почты. Для каждой подписки отображается тикер компании и список условий, при выполнении которых должно быть отправлено уведомление.

При необходимости пользователь может удалить любую подписку одним нажатием кнопки или изменить статус подписки нажатием кнопки активировать/деактивировать (Рисунок 11). После удаления запись автоматически удаляется из базы данных и больше не участвует в проверке условий уведомления.

Использование механизма Magic Link позволило отказаться от создания отдельной системы регистрации и авторизации пользователей, сохранив при этом возможность безопасного управления созданными подписками.

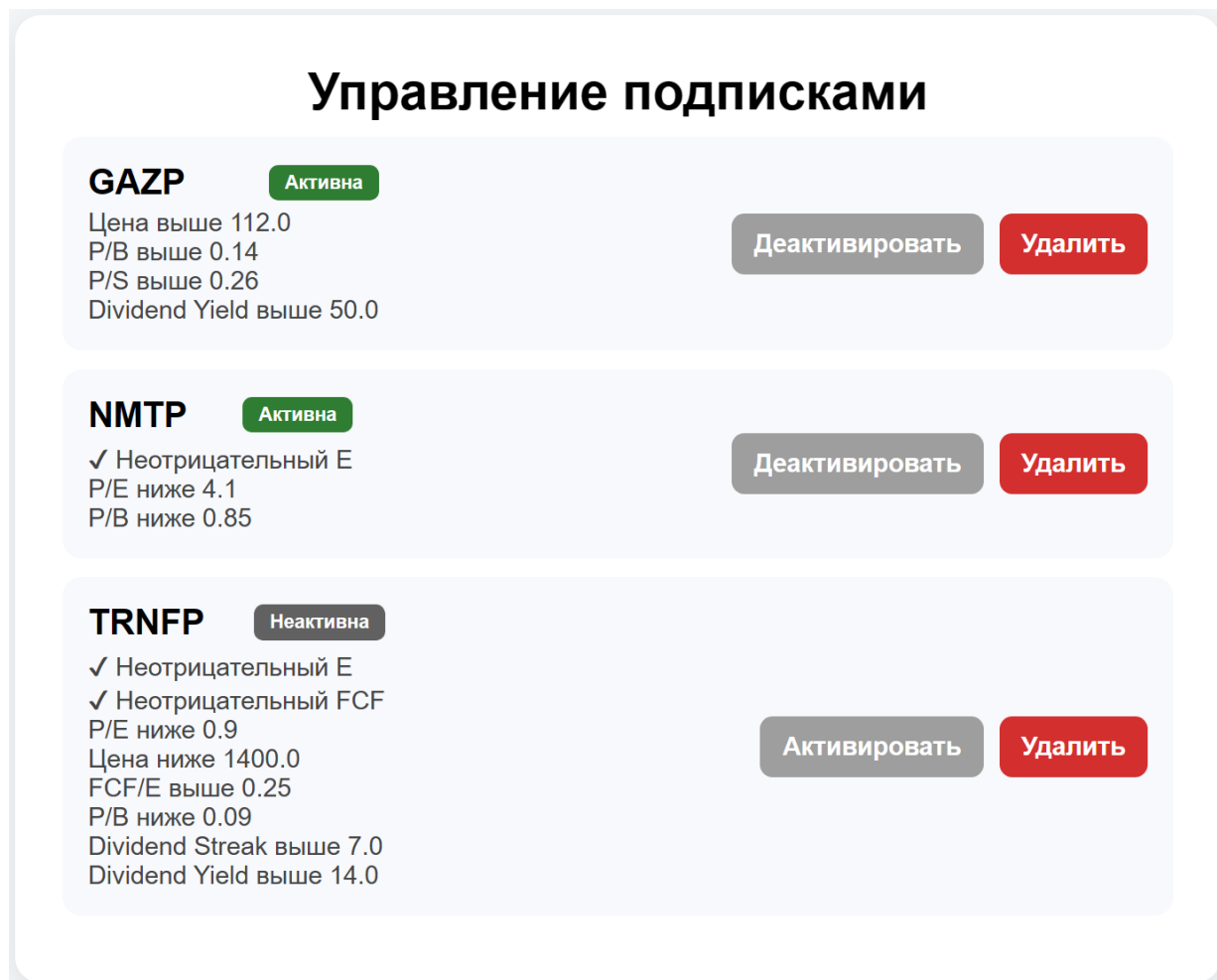


Рисунок 11 – Страница управления подписками

Логика реализации:

Для повышения удобства использования в системе реализован механизм управления подписками через персональную ссылку. После создания подписки пользователю не требуется проходить дополнительную регистрацию или авторизацию. Вместо этого система формирует уникальную ссылку, содержащую специальный токен, связанный с конкретным пользователем. Данная ссылка включается в текст электронного письма и позволяет перейти на страницу управления подписками. Формирование ссылки выполняется в момент отправки уведомления пользователю (Листинг 16).

```

cursor.execute("""
    select manage_token
    from users
    where id = %s
""", (user_id,))
manage_token = cursor.fetchone()[0]
manage_link = (
    f"http://127.0.0.1:5000/manage"
    f"?token={manage_token}"
)
send_email(
    email,
    ticker,
    price,
    pe,
    pb,
    conditions,
    manage_link
)

```

Листинг 16 – Формирование ссылки управления подписками

После перехода по ссылке приложение получает токен из параметров URL и выполняет поиск соответствующего пользователя в базе данных. Если пользователь найден, производится загрузка всех созданных им подписок, включая тикер акции и список условий срабатывания уведомления. Полученные данные используются для формирования страницы управления подписками (Листинг 17).

```

token = request.args.get("token")
cursor.execute("""
    select id, email

```

```

from users
where manage_token = %s
"""', (token,))
user = cursor.fetchone()
...
cursor.execute("""
select
    sub.id,
    st.ticker,
    st.name,
    sub.conditions_json
from subscriptions sub
join stocks st
    on sub.stock_id = st.id
where sub.user_id = %s
order by st.ticker
""", (user_id,))
rows = cursor.fetchall()
subs = []
for row in rows:
    subs.append({
        "id": row[0],
        "ticker": row[1],
        "name": row[2],
        "conditions": row[3]
    })

```

Листинг 17 – Получение списка подписок пользователя

На странице управления пользователь может удалить любую активную подписку или деактивировать/активировать ее. При нажатии кнопки удаления

идентификатор подписки и токен пользователя передаются в обработчик запроса. Перед выполнением операции система повторно проверяет принадлежность токена пользователю, после чего удаляет только те записи, которые связаны с данным пользователем. Такой подход предотвращает возможность удаления чужих подписок при изменении параметров запроса (Листинг 18).

```
sub_id = request.args.get("id")
token = request.args.get("token")
cursor.execute("""
    select id
    from users
    where manage_token = %s
""", (token,))
user = cursor.fetchone()
...
user_id = user[0]
cursor.execute("""
    delete from subscriptions
    where id = %s
    and user_id = %s
""", (
    sub_id,
    user_id
))
conn.commit()
return redirect(
    f"/manage?token={token}"
)
```

Листинг 18 – Удаление подписки

2.7 Тестирование программы

Данный раздел содержит последовательность действий для проверки работоспособности основных функций разработанной системы мониторинга акций.

1) Запуск приложения

- Предварительные условия: запущен сервер PostgreSQL и настроены параметры подключения к базе данных.
- Действие: запустить веб-приложение командой:
`python invest_app.py`
Открыть в браузере адрес:
`http://127.0.0.1:5000`
- Ожидаемый результат: отображается главная страница приложения со списком доступных акций и элементами управления.

2) Работа таблицы акций

- Действие: после открытия главной страницы обратить внимание на таблицу акций, расположенную в верхней части интерфейса.
- Ожидаемый результат: отображаются пять акций с показателями текущей цены, коэффициентов P/E, P/B и ROE.
- Действие: выполнить сортировку по любому из столбцов таблицы.
- Ожидаемый результат: строки таблицы перестраиваются в соответствии с выбранным параметром сортировки.
- Действие: нажать на тикер любой компании.
- Ожидаемый результат: происходит переход на страницу выбранной акции с загрузкой соответствующих данных.

3) Просмотр информации об акции

- Действие: выбрать любую акцию из списка, расположенного в верхней части страницы или ввести название нужной акции в поле ввода.

- Ожидаемый результат: отображается информация о выбранной компании, включая тикер, текущую стоимость акции, график цены за последний день и основные финансовые показатели.

4) Изменение периода отображения графика

- Действие: после выбора акции воспользоваться элементами выбора периода отображения данных («день», «неделя», «месяц», «год» и т.д.).
- Ожидаемый результат: график перестраивается в соответствии с выбранным временным интервалом и отображает данные только за указанный период.
- Дополнительная проверка: при отображении большого количества точек воспользоваться горизонтальным ползунком для просмотра отдельных участков графика.
- Ожидаемый результат: пользователь может перемещаться по графику без потери читаемости данных и просматривать интересующие временные интервалы.

5) Переключение отображаемого показателя

- Действие: выбрать последовательно показатели «Цена», «P/E» и «P/B».
- Ожидаемый результат: график перестраивается и отображает динамику выбранного показателя.
- Дополнительная проверка: выполнить переключение между показателями при наличии нескольких компаний в режиме сравнения.
- Ожидаемый результат: данные всех выбранных компаний отображаются корректно для каждого показателя.

6) Сравнение нескольких акций на графике

- Действие: выбрать основную акцию и добавить дополнительные акции через панель сравнения.
- Ожидаемый результат: на графике одновременно отображаются данные нескольких компаний, что позволяет визуально сравнивать динамику выбранного показателя.

- Дополнительная проверка: удалить одну из добавленных акций из списка сравнения.
- Ожидаемый результат: график обновляется и отображает данные только для оставшихся компаний.

7) Просмотр финансовых показателей компании

- Действие: открыть страницу любой акции и перейти к блоку финансовых показателей под графиком.
- Ожидаемый результат: отображаются рассчитанные показатели P/E, P/B, P/S, ROE, Net Debt/E, FCF/E, данные финансовой отчетности компании и информация о последних дивидендных выплатах.

8) Создание подписки на уведомления

- Действие: в блоке подписки указать адрес электронной почты, выбрать условие срабатывания и ввести требуемое значение. Нажать кнопку «Подписаться».
- Ожидаемый результат: информация о подписке сохраняется в базе данных и становится доступна для дальнейшего использования системой уведомлений.

9) Отправка уведомления

- Действие: дожидаться выполнения условий подписки.
- Ожидаемый результат: на указанный адрес электронной почты поступает письмо с информацией о выполнении заданных условий и ссылкой для управления подписками.

10) Управление подписками

- Действие: перейти по ссылке из письма либо открыть страницу управления подписками по индивидуальному токenu пользователя.
- Ожидаемый результат: отображается список активных подписок пользователя с указанием контролируемых условий.
- Действие: нажать кнопку удаления подписки.
- Ожидаемый результат: выбранная подписка удаляется из базы данных и больше не участвует в проверке условий уведомления.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было разработано веб-приложение для мониторинга акций российских компаний и анализа их финансовых показателей. Созданная система обеспечивает автоматизированный сбор и хранение рыночных данных, расчет инвестиционных коэффициентов, визуализацию информации и своевременное оповещение пользователей о достижении заданных условий.

В процессе выполнения работы были решены следующие задачи:

1. Спроектирована и реализована структура базы данных PostgreSQL для хранения информации об акциях, исторических котировках, финансовой отчетности компаний, дивидендных выплатах и пользовательских подписках.
2. Разработан механизм автоматизированного сбора данных с использованием открытого API Московской биржи, обеспечивающий регулярное обновление котировок акций и рыночной статистики.
3. Реализован расчет инвестиционных коэффициентов P/E, P/B, P/S, ROE, Net Debt/E, FCF/E и показателя ROE на основе данных финансовой отчетности и рыночной капитализации компаний.
4. Разработан веб-интерфейс на базе фреймворка Flask, обеспечивающий удобное отображение рыночной и финансовой информации для пользователей.
5. Реализовано построение графиков стоимости акций и инвестиционных коэффициентов с возможностью выбора временного периода отображения данных.
6. Реализован механизм сравнения нескольких акций на одном графике, позволяющий проводить сравнительный анализ инвестиционных инструментов.
7. Разработана система пользовательских подписок на уведомления с поддержкой контроля стоимости акций, коэффициентов P/E, P/B, P/S, Net Debt/E, FCF/E, значений ликвидности и дивидендных показателей.

8. Реализован механизм автоматической отправки уведомлений по электронной почте при выполнении заданных пользователем условий.
9. Разработан интерфейс управления подписками, позволяющий просматривать, изменять статус и удалять ранее созданные уведомления посредством индивидуальной ссылки доступа.
10. Проведено тестирование разработанного программного обеспечения, подтвердившее корректность работы разработанной системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Финам — официальный сайт [Электронный ресурс]. URL: <https://www.finam.ru/> (дата обращения: 25.05.2026).
- 2) Smart-Lab — инвестиционное сообщество и аналитическая платформа [Электронный ресурс]. URL: <https://smart-lab.ru/> (дата обращения: 25.05.2026).
- 3) Investemika — инвестиционный аналитический сервис [Электронный ресурс]. URL: <https://investemika.ru/> (дата обращения: 25.05.2026).
- 4) PyCharm — официальный сайт JetBrains [Электронный ресурс]. URL: <https://www.jetbrains.com/pycharm/> (дата обращения: 25.05.2026).
- 5) Visual Studio Community 2022 — официальный сайт Microsoft [Электронный ресурс]. URL: <https://visualstudio.microsoft.com/> (дата обращения: 25.05.2026).
- 6) Python — официальный сайт [Электронный ресурс]. URL: <https://www.python.org/> (дата обращения: 25.05.2026).
- 7) C++ Reference — справочная документация языка C++ [Электронный ресурс]. URL: <https://en.cppreference.com/> (дата обращения: 25.05.2026).
- 8) PostgreSQL — официальный сайт [Электронный ресурс]. URL: <https://www.postgresql.org/> (дата обращения: 25.05.2026).
- 9) MySQL — официальный сайт [Электронный ресурс]. URL: <https://www.mysql.com/> (дата обращения: 25.05.2026).
- 10) Django Templates — официальная документация Django [Электронный ресурс]. URL: <https://docs.djangoproject.com/en/stable/topics/templates/> (дата обращения: 25.05.2026).
- 11) Streamlit — официальный сайт и документация [Электронный ресурс]. URL: <https://streamlit.io/> (дата обращения: 25.05.2026).
- 12) Jinja2 — официальная документация [Электронный ресурс]. URL: <https://jinja.palletsprojects.com/> (дата обращения: 25.05.2026).

- 13) Flask — официальная документация [Электронный ресурс]. URL: <https://flask.palletsprojects.com/> (дата обращения: 25.05.2026).
- 14) Psycopg2 — документация адаптера PostgreSQL для Python [Электронный ресурс]. URL: <https://www.psycopg.org/docs/> (дата обращения: 25.05.2026).
- 15) Requests — документация библиотеки [Электронный ресурс]. URL: <https://docs.python-requests.org/> (дата обращения: 25.05.2026).
- 16) Python-dotenv — документация библиотеки [Электронный ресурс]. URL: <https://pypi.org/project/python-dotenv/> (дата обращения: 25.05.2026).
- 17) SMTP (smtplib) — документация Python [Электронный ресурс]. URL: <https://docs.python.org/3/library/smtplib.html> (дата обращения: 25.05.2026).
- 18) email.mime — документация Python [Электронный ресурс]. URL: <https://docs.python.org/3/library/email.mime.html> (дата обращения: 25.05.2026).

ПРИЛОЖЕНИЕ А

Факультет «Космический»

Кафедра «Прикладная математика, информатика и вычислительная техника»

Специальность 09.03.01 – «Информатика и вычислительная техника»

Выпускная квалификационная работа

на тему:

Разработка веб-приложения для отслеживания
привлекательности инвестиций

Выполнил: Теплов Ярослав Александрович

Руководитель: Афанасьева В. В.

Актуальность

2

Сегодня, долгосрочный инвестор в акции российских компаний сталкивается со следующими вызовами:

- необходимость осуществлять трудовую деятельность для создания материальной базы для инвестиций (реализация своего человеческого капитала)
- необходимость следить за новостным фоном (геополитический фактор)
- необходимость регулярно пересматривать состав портфеля избавляясь от переоцененных рынком акций в пользу недооцененных (ребалансировка портфеля)

Грамотное управление долгосрочным портфелем включает постоянный анализ привлекательности ценных бумаг.

Автоматизация такого анализа, снижает затраты на принятие инвестиционных решений и избавляет от упущенных возможностей.

Существующие сегодня в России приложения для инвестиций (особенно бесплатные) слабо справляются с этой задачей, имея следующие недостатки:

- недостаточный охват ключевых показателей финансовой деятельности компаний (например, часто не указывается чистый долг)
- недостаточная актуальность основных мультипликаторов (расчет p/e , p/b раз в полгода-год)
- ошибки в расчетах мультипликаторов
- невозможность гибко настроить уведомления (уведомления только по порогу цены, а не по набору показателей)

Необходимо снять с инвестора нагрузку по постоянному пересчёту мультипликаторов акций и реализовать приложение, выполняющее эту задачу в рантайм.

Своевременное уведомление о входе фундаментальных показателей в интересующий инвестора интервал позволит в тот же день рассмотреть целесообразность покупки или продажи бумаги и не упустить благоприятную возможность.

В отличие от трейдинга, долгосрочные инвестиции не требуют частых сделок, но их эффективность можно и нужно повышать, выбирая удачные моменты для покупки или заблаговременно продавая переоцененное.

Цели и задачи

3

Цель: разработать веб-приложение для мониторинга акций российских компаний и анализа их финансовых показателей для долгосрочного инвестора.

Задачи:

- Спроектировать и реализовать базу данных для хранения рыночной информации и пользовательских подписок.
- Разработать механизм автоматизированного сбора данных Московской биржи.
- Организовать хранение исторических котировок и финансовой отчетности компаний.
- Реализовать расчет инвестиционных коэффициентов.
- Разработать средства визуализации рыночных и финансовых данных.
- Реализовать сравнение нескольких акций по выбранным показателям.
- Разработать систему подписок на пользовательские условия.
- Реализовать автоматическую отправку уведомлений по электронной почте.

Методы решения поставленных задач

4

Компонент	Технология
Язык программирования	Python
Хранение данных	PostgreSQL
Backend	Flask
Frontend	HTML,CSS,JavaScript
Визуализация графиков	Chart.js
Получение данных	ISS API Московской биржи
Email уведомления	SMTP
Автоматизация	Планировщик задач Windows

Архитектура приложения

5

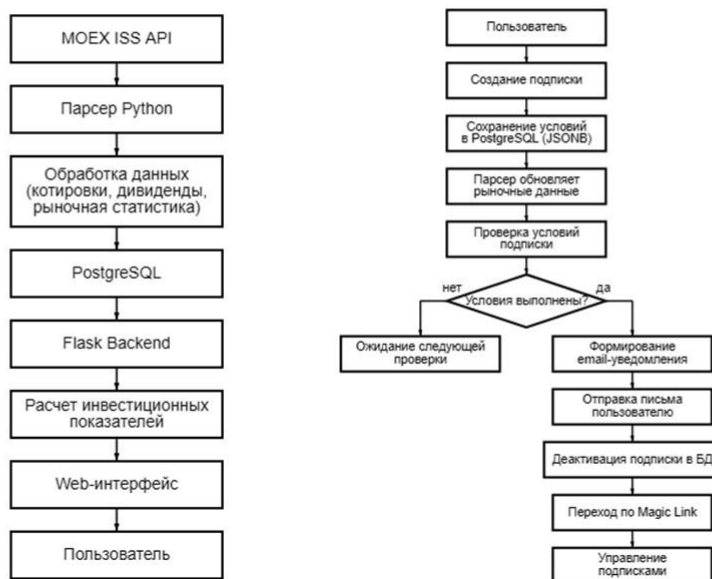
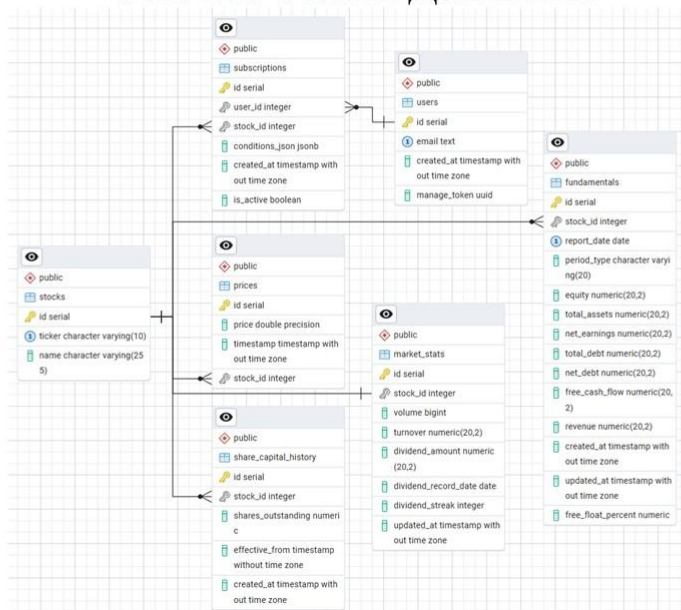
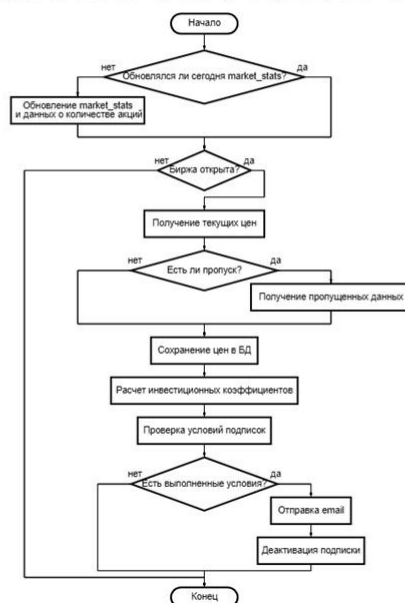


Схема базы данных

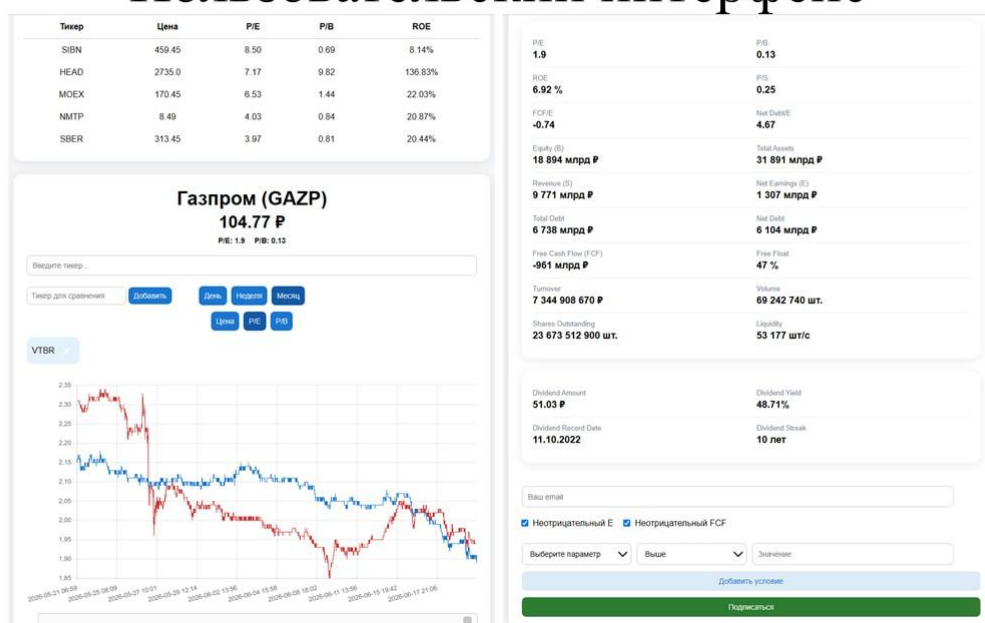
6



Алгоритм работы системы сбора данных 7



Пользовательский интерфейс 8



Система настройки уведомлений

9

example12@gmail.com

☒ Неотрицательный E ☐ Неотрицательный FCF

P/E: Ниже 1.75

P/B: Ниже 0.12

Net Debt/E: Ниже 4.65

Liquidity: Выше 40000

Dividend Streak: Выше 0

Добавить условие

Подписаться

example12@gmail.com

☐ Неотрицательный E ☐ Неотрицательный FCF

Цена: Выше 112

P/B: Выше 0.14

P/S: Выше 0.26

Dividend Yield: Выше 50

Добавить условие

Подписаться

Реализованы уведомления по:

- цене акции
- коэффициенту P/E
- коэффициенту P/B
- коэффициенту P/S
- коэффициенту FCF/E
- коэффициенту Net Debt/E
- значению ликвидности за последний день
- дивидендной доходности

Поддерживаются условия:

- выше заданного значения
- ниже заданного значения

Уведомления

10

Уведомление по акции TRNFP

Цена: 1342.2 P
P/E: 0.87
P/B: 0.07
P/S: 0.14
FCF/E: 0.34
Net Debt/E: -0.07
Liquidity: 279.54
Dividend Yield: 14.77%
Dividend Streak: 8 лет

Выполнены все условия:

- P/E ниже 0.9
- Цена ниже 1400.0
- FCF/E выше 0.25
- P/B ниже 0.09
- Dividend Streak выше 7.0
- Dividend Yield выше 14.0

Управление подписками

Система мониторинга MOEX

TRNFP

Неактивна

✓ Неотрицательный E
✓ Неотрицательный FCF
P/E ниже 0.9
Цена ниже 1400.0
FCF/E выше 0.25
P/B ниже 0.09
Dividend Streak выше 7.0
Dividend Yield выше 14.0

Активировать

Удалить

При выполнении всех заданных условий пользователю автоматически отправляется уведомление на электронную почту.

После отправки уведомления статус подписки в базе данных изменяется на «неактивна», что предотвращает повторную отправку уведомлений по тем же условиям.

Управление подписками

11

Управление подписками

GAZP Активна

Цена выше 112.0
P/B выше 0.14
P/S выше 0.26
Dividend Yield выше 50.0

Деактивировать Удалить

NMTP Активна

✓ Неотрицательный E
P/E ниже 4.1
P/B ниже 0.85

Деактивировать Удалить

TRNFP Неактивна

✓ Неотрицательный E
✓ Неотрицательный FCF
P/E ниже 0.9
Цена ниже 1400.0
FCF/E выше 0.25
P/B ниже 0.09
Dividend Streak выше 7.0
Dividend Yield выше 14.0

Активировать Удалить



Полученные результаты

12

В ходе работы:

- Разработана база данных для хранения котировок акций, финансовой отчетности, дивидендных выплат и пользовательских подписок.
- Реализован автоматизированный сбор и обновление рыночных данных Московской биржи.
- Реализован расчет инвестиционных коэффициентов P/E, P/B, P/S, ROE, Net Debt/E, FCF/E на основе финансовой отчетности компаний.
- Разработан веб-интерфейс для отображения рыночных данных, финансовых показателей и графиков.
- Реализовано сравнение нескольких акций по выбранным показателям на одном графике.
- Реализована система подписок на достижение заданных значений цены акции, коэффициентов P/E, P/B, P/S, Net Debt/E, FCF/E, значений ликвидности и дивидендных показателей.
- Реализована автоматическая отправка уведомлений по электронной почте при выполнении заданных условий.
- Реализован механизм управления подписками с использованием индивидуальных ссылок.

Спасибо за внимание!

ПРИЛОЖЕНИЕ Б

УТВЕРЖДАЮ

Зав. кафедрой К-3 МФ

_____ А.В.Корольков.

«__» _____ 2026 г.

Приложение для отслеживания привлекательности инвестиций

Спецификация

ЛИСТ УТВЕРЖДЕНИЯ

643.МГУЛ.090301-84-ЛУ

Студент

_____ Я. А. Теплов

«__» _____ 2026 г.

Руководитель

_____ В. В. Афанасьева

«__» _____ 2026 г.

Нормоконтролер

_____ А.В. Чернышов

«__» _____ 2026 г.

УТВЕРЖДЕН

643.МГУЛ.090301-84-ЛУ

Приложение для отслеживания привлекательности инвестиций

Спецификация

643.МГУЛ.090301-84

Листов 3

643.МГУЛ.090301-84

Обозначение	Наименование	Примечание
	<u>Документация</u>	
643.МГУЛ.090301-84-ЛУ	Приложение для	Размножать
	отслеживания	по
	привлекательности	указанию
	инвестиций	
	Лист утверждения	
643.МГУЛ.090301-84 12 01	Приложение для	
	отслеживания	
	привлекательности	
	инвестиций	
	Текст программы	
643.МГУЛ.090301-84 12 01-ЛУ	Приложение для	Размножать
	отслеживания	по
	привлекательности	указанию
	инвестиций	
	Текст программы	
	Лист утверждения	

73

УТВЕРЖДАЮ

Зав. кафедрой К-3 МФ

_____ А.В.Корольков.

«__»_____2026 г.

Приложение для отслеживания привлекательности инвестиций

Текст программы

ЛИСТ УТВЕРЖДЕНИЯ

643.МГУЛ.090301-84 12 01-ЛУ

Студент

_____ Я. А. Теплов

«__»_____2026 г.

Руководитель

_____ В. В. Афанасьева

«__»_____2026 г.

Нормоконтролер

_____ А.В. Чернышов

«__»_____2026 г.

УТВЕРЖДЕН

643.МГУЛ.090301-84 12 01-ЛУ

Приложение для отслеживания привлекательности инвестиций

Текст программы

643.МГУЛ.090301-84 12 01

Листов 57

Файл invest_app.py

```

from flask import Flask, render_template, request, redirect
import json
import psycopg2

app = Flask(__name__)

def get_connection():
    return psycopg2.connect(
        dbname="invest_db",
        user="postgres",
        password="1234",
        host="localhost",
        port="5432"
    )

def format_billions(value):
    if value is None:
        return "—"

    return f"{round(float(value) / 1_000_000_000):.} ".replace(", ", " ") + " млрд ₽"

def format_number(value):
    if value is None:
        return "—"

    return f"{round(float(value)):} ".replace(", ", " ")

@app.route("/manage")
def manage():
    token = request.args.get("token")

    if not token:
        return "Нет токена"

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        select id, email
        from users
        where manage_token = %s
        """, (token,))

    user = cursor.fetchone()

    if not user:
        cursor.close()
        conn.close()

        return "Неверная ссылка"

    user_id = user[0]
    email = user[1]

    cursor.execute("""
        select
            sub.id,
            st.ticker,
            st.name,

```

```

        sub.conditions_json,
        sub.is_active
    from subscriptions sub
    join stocks st
    on sub.stock_id = st.id
    where sub.user_id = %s
    order by st.ticker
    """ , (user_id,))

rows = cursor.fetchall()

subs = []

for row in rows:
    subs.append({
        "id": row[0],
        "ticker": row[1],
        "name": row[2],
        "conditions": row[3],
        "is_active": row[4]
    })

cursor.close()
conn.close()

return render_template(
    "manage.html",
    subs=subs,
    token=token,
    email=email
)

@app.route("/delete_subscription")
def delete_subscription():

    sub_id = request.args.get("id")
    token = request.args.get("token")

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        select id
        from users
        where manage_token = %s
    """ , (token,))

    user = cursor.fetchone()

    if not user:

        cursor.close()
        conn.close()

        return "Неверный токен"

    user_id = user[0]

    cursor.execute("""
        delete from subscriptions
        where id = %s
        and user_id = %s
    """ , (
        sub_id,
        user_id

```

```

))

conn.commit()

cursor.close()
conn.close()

return redirect(f"/manage?token={token}")

@app.route("/subscribe", methods=["POST"])
def subscribe():
    ticker = request.form["ticker"].upper()

    email = request.form["email"]
    positive_e = "positiveE" in request.form
    positive_fcf = "positiveFCF" in request.form
    metrics = request.form.getlist(
        "metric[]"
    )

    operators = request.form.getlist(
        "operator[]"
    )

    values = request.form.getlist(
        "value[]"
    )

    conditions_json = []
    for metric, operator, value in zip(
        metrics,
        operators,
        values
    ):

        conditions_json.append({
            "metric": metric,
            "operator": operator,
            "value": float(value)
        })

    subscription_data = {
        "conditions": conditions_json,
        "positive_e": positive_e,
        "positive_fcf": positive_fcf
    }
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        insert into users(email)
        values(%)s
        on conflict(email) do nothing
    """, (email,))

    cursor.execute("""
        select id
        from users
        where email = %s
    """, (email,))

    user_id = cursor.fetchone()[0]

    cursor.execute("""

```

643.MFYJ.090301-84 12 01

```

        select id
        from stocks
        where ticker = %s
        """ , (ticker,))

stock_id = cursor.fetchone()[0]

cursor.execute("""
    insert into subscriptions
    (
        user_id,
        stock_id,
        conditions_json,
        is_active
    )
    values (%s, %s, %s, TRUE)
    """, (
        user_id,
        stock_id,
        json.dumps(subscription_data)
    ))

conn.commit()

cursor.close()
conn.close()

return redirect(
    f'/?ticker={ticker}&subscribed=1"
)

@app.route("/activate_subscription")
def activate_subscription():

    sub_id = request.args.get("id")

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        select is_active
        from subscriptions
        where id = %s
        """, (sub_id,))

    row = cursor.fetchone()

    if not row:
        conn.close()
        return "not found", 404

    current_active = row[0]

    new_active = not current_active

    cursor.execute("""
        update subscriptions
        set is_active = %s
        where id = %s
        """, (new_active, sub_id))

    conn.commit()
    cursor.close()
    conn.close()

    return redirect(request.referrer)

```

```

@app.route("/")
def home():

    ticker = request.args.get(
        "ticker",
        "SBER"
    ).upper()

    period = request.args.get(
        "period",
        "day"
    )

    metric = request.args.get(
        "metric",
        "price"
    ).lower()

    compare = request.args.get(
        "compare",
        ""
    )

    compare_tickers = [
        t.strip().upper()
        for t in compare.split(",")
        if t.strip()
    ]

    all_tickers = [ticker]

    for t in compare_tickers:
        if t not in all_tickers:
            all_tickers.append(t)

    subscribed = request.args.get(
        "subscribed"
    )

    conn = get_connection()
    cur = conn.cursor()

    cur.execute("""
        select
            id,
            ticker,
            name
        from stocks
        where ticker = %s
    """, (ticker,))

    stock = cur.fetchone()

    if not stock:

        cur.close()
        conn.close()

        return "нет такого тикера"

    stock_id = stock[0]
    stock_ticker = stock[1]
    stock_name = stock[2]

    stock_display_name = (
        f"{stock_name} ({stock_ticker})"

```

```

)

cur.execute("""
    select max(timestamp)
    from prices
    where stock_id = %s
""", (stock_id,))

latest_timestamp = cur.fetchone()[0]

if not latest_timestamp:

    cur.close()
    conn.close()

    return "нет данных по акции"

cur.execute("""
    select
        f.equity,
        f.total_assets,
        f.net_earnings,
        f.total_debt,
        f.net_debt,
        f.free_cash_flow,
        f.revenue,
        f.free_float_percent,
        m.volume,
        m.turnover,
        sch.shares_outstanding
    from fundamentals f

    left join market_stats m
        on f.stock_id = m.stock_id

    left join (
        select distinct on (stock_id)
            stock_id,
            shares_outstanding
        from share_capital_history
        order by stock_id, effective_from desc
    ) sch
        on f.stock_id = sch.stock_id

    where f.stock_id = %s

    order by f.report_date desc

    limit 1
""", (stock_id,))

fundamentals = cur.fetchone()

cur.execute("""
    select
        dividend_amount,
        dividend_record_date,
        dividend_streak
    from market_stats
    where stock_id = %s
""", (stock_id,))

dividend_data = cur.fetchone()

day_interval = "1 day"

```

643.МГҮЛ.090301-84 12 01

```

cur.execute("""
    select price
    from prices
    where stock_id = %s
    and timestamp >= %s::timestamp - interval %s
    order by timestamp
""", (
    stock_id,
    latest_timestamp,
    day_interval
))

```

```
day_data = cur.fetchall()
```

```

day_prices = [
    float(x[0])
    for x in day_data
]

```

```

interval = {
    "day": "1 day",
    "week": "7 days",
    "month": "30 days"
}.get(period, "1 day")

```

```

cur.execute("""
    select
        price,
        timestamp
    from prices
    where stock_id = %s
    and timestamp >= %s::timestamp - interval %s
    order by timestamp
""", (
    stock_id,
    latest_timestamp,
    interval
))

```

```
data = cur.fetchall()
```

```
# история количества акций
```

```

cur.execute("""
    select
        effective_from,
        shares_outstanding
    from share_capital_history
    where stock_id = %s
    order by effective_from
""", (stock_id,))

```

```
share_history = cur.fetchall()
```

```
# история фундамента
```

```

cur.execute("""
    select
        report_date,
        equity,
        net_earnings
    from fundamentals
    where stock_id = %s
    order by report_date
""", (stock_id,))

```

```
fundamentals_history = cur.fetchall()
```

643.MFYJL.090301-84 12 01

```

colors = [
    "#1976d2",
    "#d32f2f",
    "#2e7d32",
    "#f57c00",
    "#7b1fa2",
    "#455a64"
]
chart_datasets = []
all_tickers = [ticker] + compare_tickers
timestamps = [
    x[1].strftime("%Y-%m-%d %H:%M")
    for x in data
]
prices = []
pe_values = []
pb_values = []
for dataset_index, current_ticker in enumerate(all_tickers):
    cur.execute("""
        select id
        from stocks
        where ticker = %s
        """, (current_ticker,))
    row = cur.fetchone()
    if not row:
        continue
    current_stock_id = row[0]
    cur.execute("""
        select max(timestamp)
        from prices
        where stock_id = %s
        """, (current_stock_id,))
    current_latest_timestamp = cur.fetchone()[0]
    if not current_latest_timestamp:
        continue
    cur.execute("""
        select
            price,
            timestamp
        from prices
        where stock_id = %s
        and timestamp >= %s::timestamp - interval %s
        order by timestamp
        """, (
            current_stock_id,
            current_latest_timestamp,
            interval
        ))
    current_data = cur.fetchall()
    cur.execute("""
        select
            effective_from,
            shares_outstanding
        from share_capital_history
        where stock_id = %s
        order by effective_from
        """, (current_stock_id,))
    current_share_history = cur.fetchall()
    cur.execute("""
        select
            report_date,
            equity,
            net_earnings
        from fundamentals
        where stock_id = %s
        order by report_date
    """)

```

643.MFYJL.090301-84 12 01

```

"", (current_stock_id,))
current_fund_history = cur.fetchall()
current_prices = []
current_pe = []
current_pb = []
share_index = 0
fund_index = 0
current_shares = None
current_equity = None
current_net_earnings = None
for price, ts in current_data:
    price = float(price)
    while (
        share_index < len(current_share_history)
        and current_share_history[share_index][0] <= ts
    ):
        current_shares = float(
            current_share_history[share_index][1]
        )
        share_index += 1
    while (
        fund_index < len(current_fund_history)
        and current_fund_history[fund_index][0] <= ts.date()
    ):
        current_equity = current_fund_history[fund_index][1]
        current_net_earnings = current_fund_history[fund_index][2]
        fund_index += 1
    current_prices.append(price)
    market_cap = None
    if current_shares:
        market_cap = (
            price * current_shares
        )
    pe = None
    pb = None
    if (
        market_cap
        and current_net_earnings
        and current_net_earnings != 0
    ):
        pe = round(
            market_cap
            / float(current_net_earnings),
            2
        )
    if (
        market_cap
        and current_equity
        and current_equity != 0
    ):
        pb = round(
            market_cap
            / float(current_equity),
            2
        )

    current_pe.append(pe)
    current_pb.append(pb)

if current_ticker == ticker:

    prices = current_prices
    pe_values = current_pe
    pb_values = current_pb

```

643.MFYJ.090301-84 12 01

```

if metric == "price":
    values = current_prices
elif metric == "pe":
    values = current_pe

else:

    values = current_pb

chart_datasets.append({
    "label": current_ticker,
    "data": values,
    "borderColor": colors[
        dataset_index % len(colors)
    ],
    "borderWidth": 2,
    "pointRadius": 0,
    "pointHoverRadius": 4,
    "fill": False,
    "tension": 0
})

chart_values = prices

if len(prices) >= 2:

    change = round(
        prices[-1] - prices[0],
        2
    )

    change_percent = round(
        (
            change / prices[0]
        ) * 100,
        2
    )

else:

    change = 0
    change_percent = 0

last_price = (
    prices[-1]
    if prices
    else 0
)

cur.execute("""
select
    equity,
    total_assets,
    net_earnings,
    total_debt,
    net_debt,
    free_cash_flow,
    revenue,
    free_float_percent
from fundamentals
where stock_id = %s
order by report_date desc
limit 1
""", (stock_id,))

fund_row = cur.fetchone()

```

```

cur.execute("""
select
    volume,
    turnover,
    dividend_amount,
    dividend_record_date,
    dividend_streak
from market_stats
where stock_id = %s
""", (stock_id,))

market_row = cur.fetchone()

cur.execute("""
select shares_outstanding
from share_capital_history
where stock_id = %s
order by effective_from desc
limit 1
""", (stock_id,))

shares_row = cur.fetchone()

if fund_row:

    equity = fund_row[0]
    total_assets = fund_row[1]
    net_earnings = fund_row[2]
    total_debt = fund_row[3]
    net_debt = fund_row[4]
    free_cash_flow = fund_row[5]
    revenue = fund_row[6]
    free_float_percent = fund_row[7]

else:

    equity = None
    total_assets = None
    net_earnings = None
    total_debt = None
    net_debt = None
    free_cash_flow = None
    revenue = None
    free_float_percent = None

if market_row:

    volume = market_row[0]
    turnover = market_row[1]

    dividend_amount = market_row[2]
    dividend_record_date = market_row[3]
    dividend_streak = market_row[4]

else:

    volume = None
    turnover = None

    dividend_amount = None
    dividend_record_date = None
    dividend_streak = None

shares_outstanding = (
    shares_row[0]
    if shares_row

```

```

    else None
)

if dividend_amount and prices:

    dividend_yield = round(
        (
            float(dividend_amount)
            / float(prices[-1])
        ) * 100,
        2
    )

else:

    dividend_yield = None

market_cap = (
    last_price * float(shares_outstanding)
    if last_price and shares_outstanding
    else None
)

p_e = (
    round(
        market_cap / float(net_earnings),
        2
    )
    if market_cap
    and net_earnings
    and float(net_earnings) != 0
    else "—"
)

p_b = (
    round(
        market_cap / float(equity),
        2
    )
    if market_cap
    and equity
    and float(equity) != 0
    else "—"
)

roe = (
    round(
        float(net_earnings)
        / float(equity)
        * 100,
        2
    )
    if net_earnings
    and equity
    and float(equity) != 0
    else "—"
)

p_s = (
    round(
        market_cap / float(revenue),
        2
    )
    if market_cap
    and revenue

```

```

    and float(revenue) != 0
    else ""
)

fcf_e = (
    round(
        float(free_cash_flow)
        / float(net_earnings),
        2
    )
    if free_cash_flow
    and net_earnings
    and float(net_earnings) != 0
    else ""
)

net_debt_e = (
    round(
        float(net_debt)
        / float(net_earnings),
        2
    )
    if net_debt
    and float(net_debt) >= 0
    and net_earnings
    and float(net_earnings) != 0
    else ""
)

liquidity = (
    round(
        float(volume)
        * float(free_float_percent)
        / 61200,
        2
    )
    if volume
    and free_float_percent
    else None
)

cur.execute("""
select
    f.net_earnings,
    f.equity,
    sch.shares_outstanding
from fundamentals f
left join (
    select distinct on (stock_id)
        stock_id,
        shares_outstanding
    from share_capital_history
    order by stock_id, effective_from desc
) sch
on sch.stock_id = f.stock_id
where f.stock_id = %s
""", (stock_id,))

fundamentals = cur.fetchone()

pe_ratio = None
pb_ratio = None

if fundamentals and last_price:
    net_earnings = fundamentals[0]

```

643.MFYJL.090301-84 12 01

```

equity = fundamentals[1]
shares_outstanding = fundamentals[2]

if shares_outstanding:

    market_cap = (
        last_price
        * float(shares_outstanding)
    )

    if net_earnings and net_earnings != 0:

        pe_ratio = round(
            market_cap
            / float(net_earnings),
            2
        )

    if equity and equity != 0:

        pb_ratio = round(
            market_cap
            / float(equity),
            2
        )

cur.execute("""
select
    ticker,
    name
from stocks
order by ticker
""")

tickers = [
    {
        "ticker": row[0],
        "label": (
            f"{row[1]} ({row[0]})"
            if row[1]
            else row[0]
        )
    }
    for row in cur.fetchall()
]

cur.execute("""
select
    s.ticker,
    p.price,

    case
        when f.net_earnings is not null
            and f.equity is not null
            and f.equity != 0
        then round(
            (
                f.net_earnings
                / f.equity
                * 100
            )::numeric,
            2
        )
        else null
    end as roe,

```

643.MFYJL.090301-84 12 01

```

case
  when f.net_earnings is not null
    and f.net_earnings != 0
    and sch.shares_outstanding is not null
  then round(
    (
      (
        p.price * sch.shares_outstanding
      ) / f.net_earnings
    )::numeric,
    2
  )
  else null
end as pe,

case
  when f.equity is not null
    and f.equity != 0
    and sch.shares_outstanding is not null
  then round(
    (
      (
        p.price * sch.shares_outstanding
      ) / f.equity
    )::numeric,
    2
  )
  else null
end as pb

from stocks s

join (
  select distinct on (stock_id)
    stock_id,
    price
  from prices
  order by stock_id, timestamp desc
) p
on p.stock_id = s.id

left join (
  select distinct on (stock_id)
    stock_id,
    net_earnings,
    equity
  from fundamentals
  order by stock_id, report_date desc
) f
on f.stock_id = s.id

left join (
  select distinct on (stock_id)
    stock_id,
    shares_outstanding
  from share_capital_history
  order by stock_id, effective_from desc
) sch
on sch.stock_id = s.id

order by pe desc
""")

market_table = cur.fetchall()

stocks_table = []

```

643.MFYJ.090301-84 12 01

```

for row in market_table:
    stocks_table.append({
        "ticker": row[0],
        "price": row[1],
        "roe": row[2],
        "pe": row[3],
        "pb": row[4]
    })
cur.close()
conn.close()
equity_display = format_billions(equity)
total_assets_display = format_billions(total_assets)
revenue_display = format_billions(revenue)
net_earnings_display = format_billions(net_earnings)
total_debt_display = format_billions(total_debt)
net_debt_display = format_billions(net_debt)
free_cash_flow_display = format_billions(free_cash_flow)
liquidity_display = format_number(liquidity)
turnover_display = format_number(turnover)
volume_display = format_number(volume)
shares_outstanding_display = format_number(shares_outstanding)
dividend_amount_display = (
    f"{dividend_amount} P"
    if dividend_amount is not None
    else "—"
)
dividend_yield_display = (
    f"{dividend_yield}%"
    if dividend_yield is not None
    else "—"
)
dividend_record_date_display = (
    dividend_record_date.strftime("%d.%m.%Y")
    if dividend_record_date
    else "—"
)
dividend_streak_display = (
    f"{dividend_streak} лет"
    if dividend_streak is not None
    else "—"
)
return render_template(
    "index.html",
    ticker=ticker,
    stock_name=stock_name,
    stock_display_name=stock_display_name,
    prices=prices,
    chart_values=json.dumps(chart_values),
    timestamps=json.dumps(timestamps),
    tickers=tickers,
    last_price=last_price,
    change=change,
    change_percent=change_percent,
    period=period,
    metric=metric,
    subscribed=subscribed,
    pe_ratio=pe_ratio,
    pb_ratio=pb_ratio,
    p_e=p_e,
    p_b=p_b,
    roe=roe,
    p_s=p_s,
    fcf_e=fcf_e,
    net_debt_e=net_debt_e,
    liquidity=liquidity_display,
    equity=equity_display,

```

643.MFYJL.090301-84 12 01

```

total_assets=total_assets_display,
revenue=revenue_display,
net_earnings=net_earnings_display,
total_debt=total_debt_display,
net_debt=net_debt_display,
free_cash_flow=free_cash_flow_display,
free_float_percent=free_float_percent,
volume=volume_display,
turnover=turnover_display,
shares_outstanding=shares_outstanding_display,
dividend_amount=dividend_amount_display,
dividend_yield=dividend_yield_display,
dividend_record_date=dividend_record_date_display,
dividend_streak=dividend_streak_display,
liquidity_value=liquidity,
dividend_yield_value=dividend_yield,
dividend_streak_value=dividend_streak,
market_table=market_table,
stocks_table=stocks_table,
chart_datasets=chart_datasets
)
if __name__ == "__main__":
    app.run(debug=True)

```

Файл parser.py

```

import os
import sys
import requests
import psycopg2
import traceback
import json
from datetime import datetime, timedelta
from psycopg2.extras import execute_values
from email_utils import send_email
conn = psycopg2.connect(
    dbname="invest_db",
    user="postgres",
    password="1234",
    host="localhost",
    port="5432"
)
cursor = conn.cursor()
LOCK_FILE = "parser.lock"
URL = (
    "https://iss.moex.com/iss/"
    "engines/stock/markets/shares/"
    "boards/tqbr/securities.json"
)
CANDLES_URL = (
    "https://iss.moex.com/iss/"
    "engines/stock/markets/shares/"
    "boards/tqbr/securities"
)
if os.path.exists(LOCK_FILE):
    print("парсер уже запущен")
    sys.exit()
with open(LOCK_FILE, "w") as f:
    f.write("running")

def load_stocks():
    cursor.execute("""
        select ticker, id
        from stocks
        """)
    return dict(cursor.fetchall())

def get_subscriptions(stock_id):
    cursor.execute("""
        select
            s.id,
            s.user_id,
            s.conditions_json,
            u.email,
            s.is_active
        from subscriptions s
        join users u
        on u.id = s.user_id
        where s.stock_id = %s
        """, (stock_id,))
    return cursor.fetchall()

def check_and_notify(
    stock_id,
    ticker,
    price,
    pe,
    pb,
    ps,
    net_debt_e,

```

```

fcf_e,
liquidity,
dividend_streak,
dividend_yield,
net_earnings,
free_cash_flow
):
subscriptions = get_subscriptions(stock_id)
metrics = {
    "price": price,
    "pe": pe,
    "pb": pb,
    "ps": ps,
    "net_debt_e": net_debt_e,
    "fcf_e": fcf_e,
    "liquidity": liquidity,
    "dividend_streak": dividend_streak,
    "dividend_yield": dividend_yield
}
for (
    sub_id,
    user_id,
    conditions_json,
    email,
    active
) in subscriptions:
    if not active:
        continue
    conditions = conditions_json["conditions"]
    positive_e = conditions_json.get(
        "positive_e",
        False
    )
    positive_fcf = conditions_json.get(
        "positive_fcf",
        False
    )
    all_conditions_met = True
    if positive_e:
        if (
            net_earnings is None
            or net_earnings < 0
        ):
            all_conditions_met = False
    if positive_fcf:
        if (
            free_cash_flow is None
            or free_cash_flow < 0
        ):
            all_conditions_met = False
    if not all_conditions_met:
        continue
    for condition_data in conditions:
        metric = condition_data["metric"]
        operator = condition_data["operator"]
        target = float(condition_data["value"])
        current_value = metrics.get(metric)
        if current_value is None:
            all_conditions_met = False
            break
        if operator == "above":
            if current_value <= target:
                all_conditions_met = False
                break
        elif operator == "below":
            if current_value >= target:

```

```

        all_conditions_met = False
        break
    if not all_conditions_met:
        continue
    print(
        f"сработало: {ticker} → {email}"
    )
    cursor.execute("""
        select manage_token
        from users
        where id = %s
        """, (user_id,))
    manage_token = cursor.fetchone()[0]
    manage_link = (
        f"http://127.0.0.1:5000/manage"
        f"?token={manage_token}"
    )
    send_email(
        email,
        ticker,
        price,
        pe,
        pb,
        ps,
        fcf_e,
        net_debt_e,
        liquidity,
        dividend_yield,
        dividend_streak,
        conditions,
        manage_link
    )
    cursor.execute("""
        update subscriptions
        set is_active = false
        where id = %s
        """, (sub_id,))

def fetch_all():
    stocks_map = load_stocks()
    r = requests.get(URL)
    r.raise_for_status()
    data = r.json()
    columns = data["marketdata"]["columns"]
    rows = data["marketdata"]["data"]
    idx_sec = columns.index("SECID")
    idx_last = columns.index("LAST")
    result = []
    for row in rows:
        ticker = row[idx_sec].upper()
        price = row[idx_last]
        if ticker not in stocks_map:
            continue
        if price is None:
            continue
        result.append((
            ticker,
            float(price)
        ))
    return result

def fetch_market_stats(ticker):
    url = (
        f"{CANDLES_URL}/{ticker}.json"
    )
    r = requests.get(
        url,

```

643.МГҮЛ.090301-84 12 01

```

        timeout=15
    )
    r.raise_for_status()
    data = r.json()
    market = {}
    security = {}
    if data["marketdata"]["data"]:
        market = dict(zip(
            data["marketdata"]["columns"],
            data["marketdata"]["data"][0]
        ))
    if data["securities"]["data"]:
        security = dict(zip(
            data["securities"]["columns"],
            data["securities"]["data"][0]
        ))
    return {
        "shares_outstanding":
            security.get("ISSUESIZE"),
        "volume":
            market.get("VOLTODAY"),
        "turnover":
            market.get("VALTODAY")
    }
def fetch_dividend_info(ticker):
    url = (
        f"https://iss.moex.com/iss/"
        f"securities/{ticker}/dividends.json"
    )
    try:
        r = requests.get(
            url,
            timeout=15
        )
        r.raise_for_status()
        data = r.json()
        rows = data["dividends"]["data"]
        columns = data["dividends"]["columns"]
        if not rows:
            return {
                "dividend_amount": None,
                "dividend_record_date": None,
                "dividend_streak": None
            }
        latest = dict(zip(
            columns,
            rows[-1]
        ))
        return {
            "dividend_amount":
                latest.get("value"),
            "dividend_record_date":
                latest.get(
                    "registryclosedate"
                ),
            "dividend_streak":
                len(rows)
        }
    except Exception as e:
        print(
            f"ошибка dividends {ticker}: {e}"
        )
    return {
        "dividend_amount": None,
        "dividend_record_date": None,
        "dividend_streak": None
    }

```

```

    }
def should_update_daily_stats():
    cursor.execute("""
        select max(updated_at)
        from market_stats
    """)
    last_update = cursor.fetchone()[0]
    if last_update is None:
        return True
    return (
        last_update.date()
        < datetime.now().date()
    )
def update_share_capital(
    stock_id,
    shares_outstanding
):
    if shares_outstanding is None:
        return
    cursor.execute("""
        select shares_outstanding
        from share_capital_history
        where stock_id = %s
        order by effective_from desc
        limit 1
    """, (stock_id,))
    row = cursor.fetchone()
    if row and float(row[0]) == float(shares_outstanding):
        return
    cursor.execute("""
        insert into share_capital_history (
            stock_id,
            shares_outstanding,
            effective_from
        )
        values (%s, %s, now())
    """, (
        stock_id,
        shares_outstanding
    ))
def update_market_stats():
    print("обновление market_stats...")
    stocks_map = load_stocks()
    for ticker, stock_id in stocks_map.items():
        try:
            stats = fetch_market_stats(
                ticker
            )
            dividends = fetch_dividend_info(
                ticker
            )
            update_share_capital(
                stock_id,
                stats["shares_outstanding"]
            )
            cursor.execute("""
                insert into market_stats (
                    stock_id,
                    volume,
                    turnover,
                    dividend_amount,
                    dividend_record_date,
                    dividend_streak,
                    updated_at
                )
                values (

```

643.МГҮЛ.090301-84 12 01

```

        %s,%s,%s,%s,%s,%s,%s,now()
    )
    on conflict (stock_id)
    do update set
        volume = excluded.volume,
        turnover = excluded.turnover,
        dividend_amount =
            excluded.dividend_amount,
        dividend_record_date =
            excluded.dividend_record_date,
        dividend_streak =
            excluded.dividend_streak,
        updated_at = now()
    """ , (
        stock_id,
        stats["volume"],
        stats["turnover"],
        dividends["dividend_amount"],
        dividends["dividend_record_date"],
        dividends["dividend_streak"]
    ))
    print(f"обновлено {ticker}")
except Exception as e:
    conn.rollback()
    print(
        f"ошибка market_stats "
        f"{ticker}: {e}"
    )
    traceback.print_exc()
conn.commit()
def get_last_timestamp():
    cursor.execute("""
        select max(timestamp)
        from prices
    """)
    return cursor.fetchone()[0]
def fetch_candles(
    ticker,
    from_time,
    till_time
):
    url = (
        f"{CANDLES_URL}/{ticker}"
        f"/candles.json"
    )
    result = []
    start = 0
    while True:
        params = {
            "from":
                from_time.strftime(
                    "%Y-%m-%d %H:%M:%S"
                ),
            "till":
                till_time.strftime(
                    "%Y-%m-%d %H:%M:%S"
                ),
            "interval": 1,
            "start": start
        }
        success = False
        for attempt in range(5):
            try:
                r = requests.get(
                    url,
                    params=params,

```

```

        timeout=30
    )
    r.raise_for_status()
    data = r.json()
    success = True
    break
except Exception as e:
    print(
        f'{ticker}: попытка '
        f'{attempt + 1}/5 '
        f'ошибка: {e}'
    )
if not success:
    break
candles = data["candles"]["data"]
if not candles:
    break
columns = data["candles"]["columns"]
idx_close = columns.index("close")
idx_begin = columns.index("begin")
for candle in candles:
    close_price = candle[idx_close]
    if close_price is None:
        continue
    result.append((
        float(close_price),
        datetime.fromisoformat(
            candle[idx_begin]
        ).replace(
            second=0,
            microsecond=0
        )
    ))
start += len(candles)
return result
def initial_load(days=30):
    print("начальная загрузка истории...")
    now = datetime.now().replace(
        second=0,
        microsecond=0
    )
    from_time = (
        now
        - timedelta(days=days)
    )
    stocks_map = load_stocks()
    rows = []
    for ticker, stock_id in stocks_map.items():
        print(f'загрузка {ticker}...')
        try:
            candles = fetch_candles(
                ticker,
                from_time,
                now
            )
            for price, ts in candles:
                if not market_is_open(ts):
                    continue
                rows.append((
                    stock_id,
                    price,
                    ts
                ))
        except Exception as e:
            print(
                f'ошибка {ticker}: {e}'
            )

```

```

    )
if rows:
    execute_values(
        cursor,
        """
        insert into prices
        (stock_id, price, timestamp)
        values %s
        on conflict (stock_id, timestamp)
        do nothing
        """,
        rows
    )
    conn.commit()
    print(
        f"загружено: {len(rows)}"
    )
def recover_missing_data():
    last_ts = get_last_timestamp()
    if not last_ts:
        return
    now = datetime.now().replace(
        second=0,
        microsecond=0
    )
    diff = now - last_ts
    if diff.total_seconds() < 240:
        return
    if diff.days > 3:
        return
    print(
        f"recovery c {last_ts}"
    )
    stocks_map = load_stocks()
    rows = []
    for ticker, stock_id in stocks_map.items():
        try:
            candles = fetch_candles(
                ticker,
                last_ts,
                now
            )
            for price, ts in candles:
                if not market_is_open(ts):
                    continue
                rows.append((
                    stock_id,
                    price,
                    ts
                ))
        except Exception as e:
            print(
                f"ошибка recovery "
                f"{ticker}: {e}"
            )
    if rows:
        execute_values(
            cursor,
            """
            insert into prices
            (stock_id, price, timestamp)
            values %s
            on conflict (stock_id, timestamp)
            do nothing
            """,
            Rows

```

```

    )
    conn.commit()
def save_data(items):
    now = datetime.now().replace(
        second=0,
        microsecond=0
    )
    stocks_map = load_stocks()
    for ticker, price in items:
        if ticker not in stocks_map:
            continue
        stock_id = stocks_map[ticker]
        cursor.execute("""
            insert into prices (
                stock_id,
                price,
                timestamp
            )
            values (%s, %s, %s)
            on conflict (
                stock_id,
                timestamp
            )
            do nothing
        """, (
            stock_id,
            price,
            now
        ))
        cursor.execute("""
            select
                f.net_earnings,
                f.equity,
                f.revenue,
                f.net_debt,
                f.free_cash_flow,
                f.free_float_percent,
                sch.shares_outstanding,
                ms.volume,
                ms.dividend_amount,
                ms.dividend_streak
            from fundamentals f
            left join (
                select distinct on (stock_id)
                    stock_id,
                    shares_outstanding
                from share_capital_history
                order by stock_id, effective_from desc
            ) sch
                on sch.stock_id = f.stock_id
            left join market_stats ms
                on ms.stock_id = f.stock_id
            where f.stock_id = %s
            order by f.report_date desc
            limit 1
        """, (stock_id,))
        row = cursor.fetchone()
        pe = None
        pb = None
        ps = None
        net_debt_e = None
        fcf_e = None
        liquidity = None
        dividend_streak = None
        dividend_yield = None

```

```

if row:
    net_earnings = row[0]
    equity = row[1]
    revenue = row[2]
    net_debt = row[3]
    free_cash_flow = row[4]
    free_float_percent = row[5]
    shares_outstanding = row[6]
    volume = row[7]
    dividend_amount = row[8]
    dividend_streak = row[9]
    if shares_outstanding:
        market_cap = (
            float(price)
            * float(shares_outstanding)
        )
        if (
            net_earnings
            and float(net_earnings) != 0
        ):
            pe = round(
                market_cap
                / float(net_earnings),
                2
            )
            if net_debt is not None:
                net_debt_e = round(
                    float(net_debt)
                    / float(net_earnings),
                    2
                )
            if free_cash_flow is not None:
                fcf_e = round(
                    float(free_cash_flow)
                    / float(net_earnings),
                    2
                )
            if (
                equity
                and float(equity) != 0
            ):
                pb = round(
                    market_cap
                    / float(equity),
                    2
                )
            if (
                revenue
                and float(revenue) != 0
            ):
                ps = round(
                    market_cap
                    / float(revenue),
                    2
                )
        if (
            volume
            and free_float_percent
        ):
            liquidity = round(
                float(volume)
                * float(free_float_percent)
                / 61200,

```

```

        2
    )
    if (
        dividend_amount
        and price
    ):
        dividend_yield = round(
            float(dividend_amount)
            / float(price)
            * 100,
            2
        )
    check_and_notify(
        stock_id,
        ticker,
        price,
        pe,
        pb,
        ps,
        net_debt_e,
        fcf_e,
        liquidity,
        dividend_streak,
        dividend_yield,
        net_earnings,
        free_cash_flow
    )
    conn.commit()
def market_is_open(
    check_time=None
):
    if check_time is None:
        check_time = datetime.now()
    if check_time.weekday() >= 5:
        return False
    current_minutes = (
        check_time.hour * 60
        + check_time.minute
    )
    start = 6 * 60 + 50
    end = 23 * 60 + 50
    return (
        start
        <= current_minutes
        <= end
    )
if __name__ == "__main__":
    try:
        if should_update_daily_stats():
            update_market_stats()
        if not market_is_open():
            print("рынок закрыт")
        else:
            cursor.execute("""
                select count(*)
                from prices
            """)
            if cursor.fetchone()[0] == 0:

                initial_load(days=30)
            else:
                print(
                    "проверка пропусков..."
                )
                recover_missing_data()
            data = fetch_all()

```

```
print(
    f"получено: "
    f"{len(data)} акций"
)
save_data(data)
print("сохранено в БД")
finally:
    if os.path.exists(LOCK_FILE):
        os.remove(LOCK_FILE)
```

Файл email_utils.py

```

import os
import smtplib
from dotenv import load_dotenv
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
load_dotenv()
EMAIL_LOGIN = os.getenv("EMAIL_LOGIN")
EMAIL_PASSWORD = os.getenv("EMAIL_PASSWORD")
def send_email(
    to_email,
    ticker,
    price,
    pe,
    pb,
    ps,
    fcf_e,
    net_debt_e,
    liquidity,
    dividend_yield,
    dividend_streak,
    conditions,
    manage_link
):
    subject = (
        f"{ticker}: выполнены условия уведомления"
    )
    metric_names = {
        "price": "Цена",
        "pe": "P/E",
        "pb": "P/B",
        "ps": "P/S",
        "fcf_e": "FCF/E",
        "net_debt_e": "Net Debt/E",
        "liquidity": "Liquidity",
        "dividend_yield": "Dividend Yield",
        "dividend_streak": "Dividend Streak"
    }
    operator_names = {
        "above": "выше",
        "below": "ниже"
    }
    conditions_html = ""
    for item in conditions:
        if item.get("type") == "positive_e":
            conditions_html += """
            <li>
                Неотрицательный E
            </li>
            """
            continue
        if item.get("type") == "positive_fcf":
            conditions_html += """
            <li>
                Неотрицательный FCF
            </li>
            """
            continue
        metric = metric_names.get(
            item["metric"],
            item["metric"]
        )
        operator = operator_names.get(
            item["operator"],

```

```

        item["operator"]
    )
    conditions_html += f"""
<li>
    {metric}
    {operator}
    <b>{item["value"]}</b>
</li>
"""
html = f"""
<html>
<body style="
    font-family: Arial;
    background-color: #f4f6f8;
    padding: 20px;
">
    <div style="
        background: white;
        padding: 20px;
        border-radius: 10px;
        max-width: 500px;
        margin: auto;
        box-shadow: 0 2px 10px rgba(0,0,0,0.1);
">
        <h2 style="color:#2e7d32;">
            Уведомление по акции {ticker}
        </h2>
        <div style="
            background:#f7f9fc;
            padding:15px;
            border-radius:10px;
            margin-bottom:20px;
">
            <div><b>Цена:</b> {price} ₮</div>
            <div><b>P/E:</b> {pe if pe is not None else "—"}</div>
            <div><b>P/B:</b> {pb if pb is not None else "—"}</div>
            <div><b>P/S:</b> {ps if ps is not None else "—"}</div>
            <div><b>FCF/E:</b> {fcf_e if fcf_e is not None else "—"}</div>
            <div><b>Net Debt/E:</b> {net_debt_e if net_debt_e is not None else "—"}</div>
            <div><b>Liquidity:</b> {liquidity if liquidity is not None else "—"}</div>
            <div><b>Dividend Yield:</b> {dividend_yield if dividend_yield is not None else "—"}%</div>
            <div><b>Dividend Streak:</b> {dividend_streak if dividend_streak is not None else "—"} лет</div>
        </div>
        <p>
            Выполнены все условия:
        </p>
        <ul>
            {conditions_html}
        </ul>
        <div style="margin-top: 30px;">
            <a
                href="{manage_link}"
                style="
                    background:#1565c0;
                    color:white;
                    padding:12px 18px;
                    text-decoration:none;
                    border-radius:8px;
                    display:inline-block;
                    font-weight:bold;
                "
            >
                Управление подписками
            </a>
        </div>
</hr>

```

```

        <p style="color: gray; font-size: 13px;">
            Система мониторинга МОЕХ
        </p>
    </div>
</body>
</html>
"""
msg = MIMEMultipart()
msg["Subject"] = subject
msg["From"] = EMAIL_LOGIN
msg["To"] = to_email
msg["Reply-To"] = EMAIL_LOGIN
msg.attach(MIMEText(html, "html", "utf-8"))
try:
    server = smtplib.SMTP_SSL(
        "smtp.gmail.com",
        465,
        timeout=10
    )
    server.login(
        EMAIL_LOGIN,
        EMAIL_PASSWORD
    )
    server.sendmail(
        EMAIL_LOGIN,
        to_email,
        msg.as_string()
    )
    server.quit()
    print(f'письмо отправлено: {to_email}')
except Exception as e:
    print("ошибка отправки:", e)

```

Файл index.html

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Акции</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/chartjs-plugin-zoom@2.0.1/dist/chartjs-plugin-zoom.min.js"></script>
  <style>
    body {
      font-family: Arial;
      background: #f4f6f8;
      margin: 0;
      padding: 40px 0;
      display: flex;
      flex-direction: column;
      align-items: center;
    }
    .container {
      background: white;
      padding: 25px;
      border-radius: 16px;
      width: 900px;
      box-sizing: border-box;
      box-shadow: 0 4px 20px rgba(0,0,0,0.1);
    }
    .header {
      text-align: center;
      margin-bottom: 20px;
    }
    .ticker {
      font-size: 36px;
      font-weight: bold;
      margin-bottom: 5px;
    }
    .price {
      font-size: 32px;
      font-weight: bold;
    }
    #change {
      font-size: 18px;
      margin-top: 5px;
      font-weight: bold;
    }
    input,
    select,
    button {
      padding: 10px;
      border-radius: 8px;
      border: 1px solid #ccc;
      font-size: 14px;
    }
    button {
      cursor: pointer;
      background: #1976d2;
      color: white;
      border: none;
    }
    button:hover {
      opacity: 0.9;
    }
    .dropdown {
      position: relative;
      margin-top: 15px;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="header">
      <div class="ticker">
        <div class="price">
          <div class="change">
            <div class="input">
              <div class="select">
                <div class="button">
                  <div class="button">
                    <div class="dropdown">

```

```

.dropdown input {
  width: 100%;
  box-sizing: border-box;
}
.dropdown-list {
  display: none;
  position: absolute;
  background: white;
  width: 100%;
  max-height: 200px;
  overflow-y: auto;
  border: 1px solid #ccc;
  border-radius: 8px;
  margin-top: 5px;
  z-index: 1000;
}
.dropdown-list div {
  padding: 8px;
  cursor: pointer;
}
.dropdown-list div:hover {
  background: #f0f0f0;
}
.compare-top {
  display: flex;
}
.compare-bottom {
  display: flex;
  justify-content: center;
}
.compare-container {
  width: 100%;
  margin: 20px 0;
}
.compare-row {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
  margin-bottom: 10px;
}
.compare-left {
  display: flex;
  align-items: center;
  gap: 2px;
}
.compare-left button {
  margin: 0;
  margin-left: -2px;
  padding: 8px 10px;
  border-radius: 8px;
}
.compare-input-wrapper {
  position: relative;
  width: 220px;
}
.compare-input-wrapper input {
  margin: 0;
  padding: 8px;
}
.periods-inline {
  display: flex;
  gap: 8px;
}
.metric-buttons {
  display: flex;

```

```

    gap: 8px;
  }
  #compareTags {
    display: flex;
    flex-wrap: wrap;
    gap: 8px;
    justify-content: flex-start;
  }
  .compare-tags-row {
    justify-content: flex-start !important;
  }
  button {
    white-space: nowrap;
  }
  .compare-tag {
    background: #e3f2fd;
    padding: 6px 10px;
    border-radius: 12px;
    display: flex;
    align-items: center;
    gap: 6px;
  }
  .compare-tag button {
    background: transparent;
    border: none;
    cursor: pointer;
    font-weight: bold;
  }
  #compareDropdown {
    position: absolute;
    top: 100%;
    left: 0;
    right: 0;
    z-index: 99999;
    background: white;
    border: 1px solid #ccc;
    border-radius: 8px;
  }
  #compareDropdown div {
    padding: 8px;
    cursor: pointer;
  }
  #compareDropdown div:hover {
    background: #f0f0f0;
  }
  body {
    position: relative;
  }
  .periods {
    display: flex;
    gap: 10px;
    margin-top: 20px;
    justify-content: center;
  }
  .period-btn.active {
    background: #1259a7;
  }
  canvas {
    margin-top: 30px;
  }
  .subscribe-box {
    margin-top: 35px;
    background: #f7f9fc;
    border-radius: 12px;
    padding: 20px;
  }

```

```

.subscribe-title {
  font-size: 22px;
  margin-bottom: 15px;
  font-weight: bold;
}
.subscribe-form {
  margin-top: 35px;
  display: flex;
  flex-direction: column;
  flex-wrap: wrap;
  gap: 12px;
}
.subscribe-form select {
  flex: 1;
  min-width: 180px;
}
.subscribe-form button {
  background: #2e7d32;
}
.success-message {
  position: fixed;
  top: 20px;
  right: 20px;
  background: #2e7d32;
  color: white;
  padding: 14px 20px;
  border-radius: 10px;
  font-weight: bold;
  box-shadow: 0 4px 12px rgba(0,0,0,0.2);
  z-index: 9999;
  animation: fadeOut 4s forwards;
}
#scrollbar {
  width: calc(100% - 24px);
  margin: 12px auto 0 auto;
  display: block;
  appearance: none;
  height: 6px;
  background: #fafafa;
  border-radius: 4px;
  outline: none;
}
#scrollbar::-webkit-slider-thumb {
  appearance: none;
  width: 16px;
  height: 16px;
  background: #c2c7cf;
  border-radius: 4px;
  cursor: pointer;
}
#scrollbar::-webkit-slider-thumb:hover {
  background: #b4bac3;
}
#scrollbar::-moz-range-thumb {
  width: 16px;
  height: 16px;
  background: #c2c7cf;
  border-radius: 4px;
  cursor: pointer;
}
.scrollbar-wrapper {
  width: 95%;
  margin: 0 auto;
}
#scrollbar {
  width: 100%;

```

```

}
.extra-stats {
  display: flex;
  justify-content: center;
  gap: 18px;
  margin-top: 8px;
  font-size: 14px;
  font-weight: 600;
}
.top-stocks-wrapper {
  width: 900px;
  margin: 0 auto 25px auto;
  background: white;
  border-radius: 16px;
  padding: 20px;
  box-sizing: border-box;
  box-shadow: 0 4px 20px rgba(0,0,0,0.08);
}
.top-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.top-title {
  font-size: 24px;
  font-weight: bold;
}
.top-switch {
  display: flex;
  gap: 10px;
}
.top-btn {
  background: #e9eef5;
  color: #333;
  border: none;
  padding: 8px 16px;
  border-radius: 10px;
  cursor: pointer;
  transition: 0.2s;
}
.top-btn.active {
  background: #1976d2;
  color: white;
}
.top-date {
  margin-top: 8px;
  color: gray;
  font-size: 14px;
  margin-bottom: 14px;
}
.top-stock-row {
  display: flex;
  justify-content: space-between;
  padding: 10px 12px;
  border-radius: 10px;
  cursor: pointer;
  transition: 0.15s;
}
.top-stock-row:hover {
  background: #f1f4f8;
}
.top-stock-ticker {
  font-weight: bold;
}
.top-stock-change.positive {
  color: #2e7d32;

```

```

    font-weight: bold;
  }
  .top-stock-change.negative {
    color: #c62828;
    font-weight: bold;
  }
  .fundamentals-box {
    margin-top: 24px;
    background: white;
    border-radius: 16px;
    padding: 20px;
    box-sizing: border-box;
    box-shadow: 0 4px 20px rgba(0,0,0,0.08);
  }
  .fundamentals-row {
    display: flex;
    gap: 20px;
    padding: 12px 0;
    border-bottom: 1px solid #f0f0f0;
  }
  .fundamentals-row:last-child {
    border-bottom: none;
  }
  .fundamentals-item {
    flex: 1;
  }
  .fundamentals-item.full {
    flex: 0 0 100%;
  }
  .fundamentals-label {
    font-size: 13px;
    color: #7b8794;
    margin-bottom: 4px;
  }
  .fundamentals-value {
    font-size: 18px;
    font-weight: bold;
  }
  .compare-controls {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 8px;
  }
  .stocks-table {
    width: 100%;
    border-collapse: collapse;
  }
  .stocks-table th,
  .stocks-table td {
    padding: 12px;
    text-align: center;
  }
  .stocks-table th {
    cursor: pointer;
    font-weight: bold;
    border-bottom: 2px solid #e0e0e0;
  }
  .stocks-table th:hover {
    background: #f4f6f8;
  }
  .stocks-table tr {
    cursor: pointer;
  }
  .stocks-table tbody tr:hover {
    background: #f8fafc;
  }

```

```

    }
    .positive {
      color: #2e7d32;
      font-weight: bold;
    }
    .negative {
      color: #c62828;
      font-weight: bold;
    }
    .condition-row {
      display: flex;
      gap: 10px;
      margin-top: 8px;
      width: 100%;
    }
    .condition-row input {
      flex-grow: 1;
    }
    .condition-row button {
      flex-grow: 0;
    }
    #addConditionBtn {
      background: rgba(25,118,210,0.15);
      color: #1976d2;
    }
    .remove-condition {
      background: #d32f2f !important;
      color: white !important;
      border: none;
      min-width: 40px;
    }
    .remove-condition:hover {
      background: #b71c1c !important;
    }
    .invisible-remove {
      visibility: hidden;
    }
    .filters-row {
      display: flex;
      gap: 15px;
      margin: 8px 0;
      margin-left: -5px;
    }
    .filters-row label {
      display: flex;
      align-items: center;
      gap: 5px;
      cursor: pointer;
    }
    @keyframes fadeOut {
      0% {
        opacity: 1;
      }
      70% {
        opacity: 1;
      }
      100% {
        opacity: 0;
      }
    }
  </style>
</head>
<body>
  <div class="top-stocks-wrapper">
    <table class="stocks-table" id="stocksTable">
      <thead>

```

```

<tr>
  <th onclick="sortTable(0)">Тикер</th>
  <th onclick="sortTable(1)">Цена</th>
  <th onclick="sortTable(2)">P/E</th>
  <th onclick="sortTable(3)">P/B</th>
  <th onclick="sortTable(4)">ROE</th>
</tr>
</thead>
<tbody id="stocksTableBody">
  {% for stock in stocks_table %}
  <tr onclick="selectTicker('{{ stock.ticker }})'">
    <td data-sort="{{ stock.ticker }}">
      {{ stock.ticker }}
    </td>
    <td data-sort="{{ stock.price }}">
      {{ stock.price }}
    </td>
    <td data-sort="{{ stock.pe if stock.pe is not none else -999999 }}">
      {{ stock.pe if stock.pe is not none else "—" }}
    </td>
    <td data-sort="{{ stock.pb if stock.pb is not none else -999999 }}">
      {{ stock.pb if stock.pb is not none else "—" }}
    </td>
    <td data-sort="{{ stock.roe if stock.roe is not none else -999999 }}">
      {{ stock.roe if stock.roe is not none else "—" }}%
    </td>
  </tr>
  {% endfor %}
</tbody>
</table>
</div>
<div class="container">
  {% if subscribed %}
  <div id="successMessage" class="success-message">
    Подписка успешно создана
  </div>
  {% endif %}
  <div class="header">
    <div class="ticker">
      {{ stock_name }} ({{ ticker }})
    </div>
    <div class="price">
      {{ last_price }} ₺
    </div>
    <div class="extra-stats">
      <div>
        P/E:
        {{ pe_ratio if pe_ratio is not none else "—" }}
      </div>
      <div>
        P/B:
        {{ pb_ratio if pb_ratio is not none else "—" }}
      </div>
    </div>
  </div>
  <div class="dropdown">
    <input type="text"
      autocomplete="off"
      id="search"
      placeholder="Введите тикер..."
      onfocus="showDropdown()"
      onkeyup="filterTickers()"/>
    <div id="dropdownList" class="dropdown-list">
      {% for t in tickers %}
      <div onclick="selectTicker('{{ t.ticker }})'">
        {{ t.label }}
      </div>
      </div>
    </div>
  </div>

```

```

    </div>
    {% endfor %}
  </div>
</div>
<div class="compare-container">
  <div class="compare-row compare-top" style="display: grid; grid-template-columns: 1fr auto 1fr; align-items: center;">
    <div class="compare-left" style="display: flex; gap: 0; justify-self: start; overflow: visible;">
      <div class="compare-input-wrapper">
        <input type="text"
          id="compareInput"
          placeholder="Тикер для сравнения"
          autocomplete="off"
          onfocus="showCompareDropdown()"
          onkeyup="filterCompareTickers()">
        <div id="compareDropdown" class="dropdown-list">
          {% for t in tickers %}
            <div onclick="addCompareTicker('{{ t.ticker }}', '{{ t.label }}'">
              {{ t.label }}
            </div>
          {% endfor %}
        </div>
      </div>
      <button onclick="addCompareFromInput()" style="margin-left: -28px;">Добавить</button>
    </div>
    <!-- центрированные периоды -->
    <div class="periods-inline" style="justify-self: center;">
      <button class="period-btn {% if period == 'day' %}active{% endif %}"
        onclick="changePeriod('day')">
        День
      </button>
      <button class="period-btn {% if period == 'week' %}active{% endif %}"
        onclick="changePeriod('week')">
        Неделя
      </button>
      <button class="period-btn {% if period == 'month' %}active{% endif %}"
        onclick="changePeriod('month')">
        Месяц
      </button>
    </div>
  </div></div>
</div>
<div class="compare-row compare-bottom" style="display: flex; justify-content: center; margin-top: 12px;">
  <div class="metric-buttons">
    <button class="period-btn {% if metric == 'price' %}active{% endif %}"
      onclick="changeMetric('price')">
      Цена
    </button>
    <button class="period-btn {% if metric == 'pe' %}active{% endif %}"
      onclick="changeMetric('pe')">
      P/E
    </button>
    <button class="period-btn {% if metric == 'pb' %}active{% endif %}"
      onclick="changeMetric('pb')">
      P/B
    </button>
  </div>
</div>
<div class="compare-row compare-tags-row" style="display: flex; justify-content: flex-start; margin-top: 12px;">
  <div id="compareTags"></div>
</div>
</div>
<canvas id="chart"></canvas>
<div style="margin-top: 15px;">
  <div class="scrollbar-wrapper">
    <input type="range"
      id="scrollbar"

```

```

        min="0"
        max="100"
        value="100"
        style="width: 100%;">
    </div>
</div>
<div class="fundamentals-box">
    <div class="fundamentals-row">
        <div class="fundamentals-item">
            <div class="fundamentals-label">P/E</div>
            <div class="fundamentals-value">{{ p_e }}</div>
        </div>
        <div class="fundamentals-item">
            <div class="fundamentals-label">P/B</div>
            <div class="fundamentals-value">{{ p_b }}</div>
        </div>
    </div>
    <div class="fundamentals-row">
        <div class="fundamentals-item">
            <div class="fundamentals-label">ROE</div>
            <div class="fundamentals-value">{{ roe }} %</div>
        </div>
        <div class="fundamentals-item">
            <div class="fundamentals-label">P/S</div>
            <div class="fundamentals-value">{{ p_s }}</div>
        </div>
    </div>
    <div class="fundamentals-row">
        <div class="fundamentals-item">
            <div class="fundamentals-label">FCF/E</div>
            <div class="fundamentals-value">{{ fcf_e }}</div>
        </div>
        <div class="fundamentals-item">
            <div class="fundamentals-label">Net Debt/E</div>
            <div class="fundamentals-value">{{ net_debt_e }}</div>
        </div>
    </div>
    <div class="fundamentals-row">
        <div class="fundamentals-item">
            <div class="fundamentals-label">Equity (B)</div>
            <div class="fundamentals-value">{{ equity }}</div>
        </div>
        <div class="fundamentals-item">
            <div class="fundamentals-label">Total Assets</div>
            <div class="fundamentals-value">{{ total_assets }}</div>
        </div>
    </div>
    <div class="fundamentals-row">
        <div class="fundamentals-item">
            <div class="fundamentals-label">Revenue (S)</div>
            <div class="fundamentals-value">{{ revenue }}</div>
        </div>
        <div class="fundamentals-item">
            <div class="fundamentals-label">Net Earnings (E)</div>
            <div class="fundamentals-value">{{ net_earnings }}</div>
        </div>
    </div>
    <div class="fundamentals-row">
        <div class="fundamentals-item">
            <div class="fundamentals-label">Total Debt</div>
            <div class="fundamentals-value">{{ total_debt }}</div>
        </div>
        <div class="fundamentals-item">
            <div class="fundamentals-label">Net Debt</div>
            <div class="fundamentals-value">{{ net_debt }}</div>
        </div>
    </div>

```

```

</div>
<div class="fundamentals-row">
  <div class="fundamentals-item">
    <div class="fundamentals-label">Free Cash Flow (FCF)</div>
    <div class="fundamentals-value">{{ free_cash_flow }}</div>
  </div>
  <div class="fundamentals-item">
    <div class="fundamentals-label">Free Float</div>
    <div class="fundamentals-value">{{ free_float_percent }} %</div>
  </div>
</div>
<div class="fundamentals-row">
  <div class="fundamentals-item">
    <div class="fundamentals-label">Turnover</div>
    <div class="fundamentals-value">{{ turnover }} P</div>
  </div>
  <div class="fundamentals-item">
    <div class="fundamentals-label">Volume</div>
    <div class="fundamentals-value">{{ volume }} mrt.</div>
  </div>
</div>
<div class="fundamentals-row">
  <div class="fundamentals-item">
    <div class="fundamentals-label">Shares Outstanding</div>
    <div class="fundamentals-value">{{ shares_outstanding }} mrt.</div>
  </div>
  <div class="fundamentals-item">
    <div class="fundamentals-label">Liquidity</div>
    <div class="fundamentals-value">{{ liquidity }} mrt/c</div>
  </div>
</div>
</div>
<div class="fundamentals-box">
  <div class="fundamentals-row">
    <div class="fundamentals-item">
      <div class="fundamentals-label">
        Dividend Amount
      </div>
      <div class="fundamentals-value">
        {{ dividend_amount }}
      </div>
    </div>
    <div class="fundamentals-item">
      <div class="fundamentals-label">
        Dividend Yield
      </div>
      <div class="fundamentals-value">
        {{ dividend_yield }}
      </div>
    </div>
  </div>
</div>
<div class="fundamentals-row">
  <div class="fundamentals-item">
    <div class="fundamentals-label">
      Dividend Record Date
    </div>
    <div class="fundamentals-value">
      {{ dividend_record_date }}
    </div>
  </div>
  <div class="fundamentals-item">
    <div class="fundamentals-label">
      Dividend Streak
    </div>
    <div class="fundamentals-value">
      {{ dividend_streak }}
    </div>
  </div>
</div>

```

```

    </div>
  </div>
</div>
</div>
<form class="subscribe-form"
  action="/subscribe"
  method="post">
  <input type="hidden"
    name="ticker"
    value="{{ ticker }}">
  <div class="condition-row">
    <input type="email"
      name="email"
      placeholder="Ваш email"
      required>
    <button type="button"
      class="remove-condition invisible-remove">
      ✕
    </button>
  </div class="condition-row">
  <div class="filters-row">
    <label>
      <input type="checkbox"
        id="positiveE"
        name="positiveE"
        checked>
      Неотрицательный E
    </label>
    <label>
      <input type="checkbox"
        id="positiveFCF"
        name="positiveFCF"
        checked>
      Неотрицательный FCF
    </label>
  </div>
  <div id="conditionsContainer">
    <div class="condition-row">
      <select name="metric[]">
        <option value="" selected disabled>
          Выберите параметр
        </option>
        <option value="price">Цена</option>
        <option value="pe">P/E</option>
        <option value="pb">P/B</option>
        <option value="ps">P/S</option>
        <option value="fcf_e">FCF/E</option>
        <option value="net_debt_e">Net Debt/E</option>
        <option value="liquidity">Liquidity</option>
        <option value="dividend_yield">Dividend Yield</option>
        <option value="dividend_streak">Dividend Streak</option>
      </select>
      <select name="operator[]">
        <option value="above">Выше</option>
        <option value="below">Ниже</option>
      </select>
      <input type="number"
        step="0.01"
        name="value[]"
        placeholder="Значение"
        required>
      <button type="button"
        class="remove-condition invisible-remove">
        ✕
      </button>
    </div>
  </div>

```

```

</div>
<button type="button"
  id="addConditionBtn">
  Добавить условие
</button>
<button type="submit">
  Подписаться
</button>
</form>
</div>
<script>
window.compareList = window.compareList || [];
const labels = {{ timestamps | safe }};
const chartDatasets = {{ chart_datasets | tojson | safe }};
chartDatasets.forEach(ds => {
  ds.borderWidth = 1;
  ds.pointRadius = 0;
});
const ctx = document.getElementById('chart');
const chart = new Chart(ctx, {
  type: 'line',
  data: {
    labels: labels,
    datasets: chartDatasets
  },
  options: {
    responsive: true,
    interaction: {
      intersect: false,
      mode: 'index'
    },
    plugins: {
      legend: {
        display: false
      },
      zoom: {
        pan: {
          enabled: true,
          mode: 'x',
          modifierKey: 'shift'
        },
        zoom: {
          wheel: {
            enabled: false
          },
          pinch: {
            enabled: true
          },
          mode: 'x'
        }
      }
    },
    scales: {
      x: {
        ticks: {
          maxTicksLimit: 10
        }
      }
    }
  }
});
function showDropdown() {
  document.getElementById("dropdownList").style.display = "block";
}
function filterTickers() {
  const input =

```

643.MFYJ.090301-84 12 01

```

document
.getElementById("search")
.value
.toUpperCase();
const items =
document
.getElementById("dropdownList")
.children;
for (let i = 0; i < items.length; i++) {
let txt = items[i].innerText;
items[i].style.display =
txt.toUpperCase().includes(input)
? ""
: "none";
}
}
function selectTicker(ticker) {
window.location.href = "?ticker=" + ticker;
}
let compareList = [];
function addCompareTicker(ticker, label) {
if (compareList.includes(ticker)) return;
compareList.push(ticker);
renderCompareTags();
updateCompareURL();
}
function addCompareFromInput() {
const input = document.getElementById("compareInput");
const ticker = input.value.trim().toUpperCase();
if (!ticker) return;
if (!compareList.includes(ticker)) {
compareList.push(ticker);
}
input.value = "";
renderCompareTags();
updateCompareURL();
}
function removeCompareTicker(ticker) {
compareList = compareList.filter(t => t !== ticker);
renderCompareTags();
updateCompareURL();
}
function renderCompareTags() {
const container = document.getElementById("compareTags");
container.innerHTML = "";
compareList.forEach(ticker => {
const div = document.createElement("div");
div.className = "compare-tag";
div.innerHTML = `
${ticker}
<button onclick="removeCompareTicker('${ticker}')">X</button>
`;
container.appendChild(div);
});
}
function updateCompareURL() {
const params = new URLSearchParams(window.location.search);
if (compareList.length > 0) {
params.set("compare", compareList.join(","));
} else {
params.delete("compare");
}
window.location.search = params.toString();
}
function showCompareDropdown() {
document.getElementById("compareDropdown").style.display = "block";
}

```

```

}
function filterCompareTickers() {
  const input =
    document.getElementById("compareInput").value.toUpperCase();
  const items =
    document.getElementById("compareDropdown").children;
  for (let i = 0; i < items.length; i++) {
    const txt = items[i].innerText;
    items[i].style.display =
      txt.toUpperCase().includes(input)
        ? ""
        : "none";
  }
}
function changePeriod(period) {
  window.location.href =
    "?ticker={{ ticker }}" +
    "&period=" + period +
    "&metric={{ metric }}";
}
function changeMetric(metric) {
  window.location.href =
    "?ticker={{ ticker }}&period={{ period }}&metric=" + metric;
}
document.addEventListener("click", function (e) {
  if (!e.target.closest(".dropdown")) {
    document.getElementById("dropdownList").style.display = "none";
  }
  if (!e.target.closest(".compare-input-wrapper")) {
    document.getElementById("compareDropdown").style.display = "none";
  }
});
const scrollbar = document.getElementById("scrollbar");
scrollbar.addEventListener("input", function () {
  const total = labels.length;
  const visible = Math.floor(total * 0.3);
  const start =
    Math.floor(
      (scrollbar.value / 100)
        * (total - visible)
    );
  const end = start + visible;
  chart.options.scales.x.min = start;
  chart.options.scales.x.max = end;
  chart.update();
});
function applyCompare() {
  const compare =
    document
      .getElementById("compareInput")
      .value
      .trim();
  window.location.href =
    "?ticker={{ ticker }}"
    + "&period={{ period }}"
    + "&metric={{ metric }}"
    + "&compare=" + encodeURIComponent(compare);
}
window.onload = function () {
  const urlParams = new URLSearchParams(window.location.search);
  const compare = urlParams.get("compare");
  if (compare) {
    compareList = compare
      .split(",")
      .map(x => x.trim())
      .filter(Boolean);
  }
}

```

```

    renderCompareTags();
  }
};
let currentColumn = -1;
let ascending = false;
function sortTable(columnIndex) {
  const tbody =
    document.getElementById("stocksTableBody");
  const rows =
    Array.from(tbody.querySelectorAll("tr"));
  if (currentColumn === columnIndex) {
    ascending = !ascending;
  } else {
    currentColumn = columnIndex;
    ascending = false;
  }
  rows.sort((a, b) => {
    let aValue =
      a.cells[columnIndex]
        .getAttribute("data-sort");
    let bValue =
      b.cells[columnIndex]
        .getAttribute("data-sort");
    let aNum = parseFloat(aValue);
    let bNum = parseFloat(bValue);
    if (!isNaN(aNum) && !isNaN(bNum)) {
      return ascending
        ? aNum - bNum
        : bNum - aNum;
    }
    return ascending
      ? aValue.localeCompare(bValue)
      : bValue.localeCompare(aValue);
  });
  tbody.innerHTML = "";
  rows.forEach(row => {
    tbody.appendChild(row);
  });
  rows.forEach((row, index) => {
    if (index < 5) {
      row.style.display = "";
    } else {
      row.style.display = "none";
    }
  });
});
}
document.getElementById("addConditionBtn")
  .addEventListener("click", function () {
    const container =
      document.getElementById(
        "conditionsContainer"
      );
    const rows =
      container.querySelectorAll(
        ".condition-row"
      );
    if (rows.length >= 8) {
      return;
    }
    container.insertAdjacentHTML(
      "beforeend",
      `
<div class="condition-row">
  <select name="metric[]">
    <option value="" selected disabled>

```

```

        Выберите параметр
    </option>
    <option value="price">Цена</option>
    <option value="pe">P/E</option>
    <option value="pb">P/B</option>
    <option value="ps">P/S</option>
    <option value="fcf_e">FCF/E</option>
    <option value="net_debt_e">Net Debt/E</option>
    <option value="liquidity">Liquidity</option>
    <option value="dividend_yield">Dividend Yield</option>
    <option value="dividend_streak">Dividend Streak</option>
</select>
<select name="operator[]">
    <option value="above">
        Выше
    </option>
    <option value="below">
        Ниже
    </option>
</select>
<input type="number"
    step="0.01"
    name="value[]"
    placeholder="Значение"
    required>
<button
    type="button"
    class="remove-condition">
    ×
</button>
</div>

```

```

    );
    updateMetricOptions();
});
document.addEventListener("click", function (e) {
    if (
        e.target.classList.contains(
            "remove-condition"
        )
    ) {
        e.target
            .closest(".condition-row")
            .remove();
    }
});
document.addEventListener(
    "change",
    function (e) {
        if (
            e.target.matches(
                'select[name="metric[]"]'
            )
        ) {
            fillCurrentValue(
                e.target
            );
        }
    }
);
document.addEventListener("DOMContentLoaded", () => {
    const rows =
        document.querySelectorAll(

```

643.MFYJ.090301-84 12 01

```

        "#stocksTableBody tr"
    );
    rows.forEach((row, index) => {
        if (index >= 5) {
            row.style.display = "none";
        }
    });
});
const currentPrice = {{ last_price }};
const currentPE = {{ p_e if p_e != "—" else "null" }};
const currentPB = {{ p_b if p_b != "—" else "null" }};
const currentPS = {{ p_s if p_s != "—" else "null" }};
const currentFCFE = {{ fcf_e if fcf_e != "—" else "null" }};
const currentNetDebtE = {{ net_debt_e if net_debt_e != "—" else "null" }};
const currentLiquidity = {{ liquidity_value if liquidity_value is not none else "null" }};
const currentDividendYield = {{ dividend_yield_value if dividend_yield_value != "—" else "null" }};
const currentDividendStreak = {{ dividend_streak_value if dividend_streak_value != "—" else "null" }};
const metricValues = {
    price: currentPrice,
    pe: currentPE,
    pb: currentPB,
    ps: currentPS,
    fcf_e: currentFCFE,
    net_debt_e: currentNetDebtE,
    liquidity: currentLiquidity,
    dividend_yield: currentDividendYield,
    dividend_streak: currentDividendStreak
};
function fillCurrentValue(select) {
    const row =
        select.closest(".condition-row");
    const input =
        row.querySelector(
            'input[name="value[]"]'
        );
    const value =
        metricValues[select.value];
    if (
        value !== null &&
        value !== undefined
    ) {
        input.value = value;
    }
    else {
        input.value = "";
    }
}
function updateMetricOptions() {
    const positiveE =
        document.getElementById("positiveE").checked;
    const positiveFCF =
        document.getElementById("positiveFCF").checked;
    const selects =
        document.querySelectorAll(
            'select[name="metric[]"]'
        );
    selects.forEach(select => {
        const options =
            select.querySelectorAll("option");
        options.forEach(opt => {
            const val = opt.value;
            if (!val) return;
            // P/S доступен только при отрицательном E
            if (val === "ps") {
                opt.style.display =
                    positiveE ? "none" : "";
            }
        });
    });
}

```

```

    opt.disabled =
      positiveE;
  }
  if (
    val === "pe" ||
    val === "net_debt_e"
  ) {
    opt.style.display =
      positiveE ? "" : "none";
    opt.disabled =
      !positiveE;
  }
  if (val === "fcf_e") {
    opt.style.display =
      (positiveE && positiveFCF)
        ? ""
        : "none";

    opt.disabled =
      !(positiveE && positiveFCF);
  }
});
if (
  select.options[
    select.selectedIndex
 ]?.disabled
) {
  select.value = "";
  fillCurrentValue(select);
}
});
}
function removeInvalidConditions() {
  const positiveE =
    document.getElementById("positiveE").checked;
  const positiveFCF =
    document.getElementById("positiveFCF").checked;
  document.querySelectorAll(".condition-row")
    .forEach(row => {
      const select = row.querySelector(
        'select[name="metric[]"]'
      );
      if (!select) {
        return;
      }
      const metric = select.value;
      let remove = false;
      if (positiveE && metric === "ps") {
        remove = true;
      }
      if (
        !positiveE &&
        (metric === "pe" ||
          metric === "net_debt_e")
      ) {
        remove = true;
      }
      if (
        (!positiveE && !positiveFCF) &&
        metric === "fcf_e"
      ) {
        remove = true;
      }
      if (remove) {
        row.remove();
      }
    });
}

```

```

    });
}
document.getElementById("positiveE")
.addEventListener("change", function () {
    removeInvalidConditions();
    updateMetricOptions();
});
document.getElementById("positiveFCF")
.addEventListener("change", function () {
    removeInvalidConditions();
    updateMetricOptions();
});
window.addEventListener(
    "load",
    updateMetricOptions
);
function isMetricAllowed(metric) {
    const positiveE =
        document.getElementById("positiveE").checked;
    const positiveFCF =
        document.getElementById("positiveFCF").checked;
    if (
        !positiveE &&
        (metric === "pe" || metric === "net_debt_e")
    ) {
        return false;
    }
    if (
        positiveE &&
        metric === "ps"
    ) {
        return false;
    }
    if (
        !positiveFCF &&
        metric === "fcf_e"
    ) {
        return false;
    }
    return true;
}
window.addEventListener("load", function () {
    updateMetricOptions();
});
</script>
</body>
</html>

```

Файл manage.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Управление подписками</title>
  <style>
    body {
      font-family: Arial;
      background: #f4f6f8;
      display: flex;
      justify-content: center;
      padding: 40px;
    }
    .container {
      width: 700px;
      background: white;
      padding: 30px;
      border-radius: 16px;
      box-shadow: 0 4px 20px rgba(0,0,0,0.1);
    }
    h1 {
      margin-top: 0;
      margin-bottom: 10px;
      text-align: center;
    }
    .email {
      text-align: center;
      color: gray;
      margin-bottom: 30px;
    }
    .sub {
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 16px;
      border-radius: 12px;
      background: #f7f9fc;
      margin-bottom: 12px;
    }
    .ticker {
      font-size: 22px;
      font-weight: bold;
    }
    .condition {
      color: #444;
      margin-top: 4px;
    }
    .delete-btn {
      background: #d32f2f;
      color: white;
      padding: 10px 14px;
      border-radius: 8px;
      text-decoration: none;
      font-weight: bold;
    }
    .actions {
      display: flex;
      gap: 10px;
      align-items: center;
    }
    .toggle-btn {
      background: #9e9e9e;
      color: white;
      padding: 10px 14px;
```

```

border-radius: 8px;
text-decoration: none;
font-weight: bold;
}
.toggle-btn:hover {
  opacity: 0.85;
}
.delete-btn:hover {
  opacity: 0.9;
}
.empty {
  text-align: center;
  color: gray;
  padding: 40px 0;
}
.ticker-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
}
.status {
  font-size: 12px;
  padding: 4px 10px;
  border-radius: 6px;
  font-weight: bold;
}
.status.active {
  background: #2e7d32;
  color: white;
}
.status.inactive {
  background: #616161;
  color: white;
}
</style>
</head>
<body>
<div class="container">
  <h1>Управление подписками</h1>
  {% if subs %}
  {% for sub in subs %}
  <div class="sub {% if not sub.is_active %}inactive-sub{% endif %}">
    <div>
      <div class="ticker-row">
        <div class="ticker">
          {{ sub.ticker }}
        </div>
        {% if sub.is_active %}
        <div class="status active">
          Активна
        </div>
        {% else %}
        <div class="status inactive">
          Неактивна
        </div>
        {% endif %}
      </div>
      <div class="condition">
        {% if sub.conditions.positive_e %}
        Неотрицательный E<br>
        {% endif %}
        {% if sub.conditions.positive_fcf %}
        Неотрицательный FCF<br>
        {% endif %}
        {% for c in sub.conditions.conditions %}

```

643.МГУЛ.090301-84 12 01

```

{% if c.metric == "price" %}
Цена
{% elif c.metric == "pe" %}
P/E
{% elif c.metric == "pb" %}
P/B
{% elif c.metric == "ps" %}
P/S
{% elif c.metric == "fcf_e" %}
FCF/E
{% elif c.metric == "net_debt_e" %}
Net Debt/E
{% elif c.metric == "liquidity" %}
Liquidity
{% elif c.metric == "dividend_yield" %}
Dividend Yield
{% elif c.metric == "dividend_streak" %}
Dividend Streak
{% endif %}
{% if c.operator == "above" %}
выше
{% elif c.operator == "below" %}
ниже
{% endif %}
{{ c.value }}
{% if not loop.last %}
<br>
{% endif %}
{% endfor %}
</div>
</div>
<div class="actions">
{% if sub.is_active %}
<a class="toggle-btn"
href="/activate_subscription?id={{ sub.id }}">
Деактивировать
</a>
{% else %}
<a class="toggle-btn"
href="/activate_subscription?id={{ sub.id }}">
Активировать
</a>
{% endif %}
<a class="delete-btn"
href="/delete_subscription?id={{ sub.id }}&token={{ token }}"
onclick="return confirm('Удалить подписку?')">
Удалить
</a>
</div>
</div>
{% endfor %}
{% else %}
<div class="empty">
У вас пока нет подписок
</div>
{% endif %}
</div>
</body>
</html>

```

[illegible]

УТВЕРЖДАЮ

Зав. кафедрой К-3 МФ

_____ А.В.Корольков.

«__» _____ 2026 г.

Приложение для отслеживания привлекательности инвестиций

Руководство оператора

ЛИСТ УТВЕРЖДЕНИЯ

643.МГУЛ.090301-84 34 01-ЛУ

Студент

_____ Я. А. Теплов

«__» _____ 2026 г.

Руководитель

_____ В. В. Афанасьева

«__» _____ 2026 г.

Нормоконтролер

_____ А.В. Чернышов

«__» _____ 2026 г.

УТВЕРЖДЕН

643.МГУЛ.090301-84 34 01-ЛУ

Приложение для отслеживания привлекательности инвестиций

Руководство оператора

643.МГУЛ.090301-84 34-01

Листов 8

АННОТАЦИЯ

В данном программном документе приведено руководство оператора к приложению для отслеживания привлекательности инвестиций.

СОДЕРЖАНИЕ

1	Назначение программы	4
2	Условия выполнения программы	4
3	Выполнение программы	5
4	Сообщения оператору	7

1 Назначение программы

Программа предназначена для мониторинга акций российских компаний, анализа финансовых показателей эмитентов и автоматического оповещения пользователей о достижении заданных условий.

Система обеспечивает централизованное хранение исторических котировок акций, финансовой отчетности компаний и информации о дивидендных выплатах. Пользователю предоставляется возможность просматривать динамику стоимости акций, анализировать инвестиционные коэффициенты P/E, P/B и ROE, сравнивать несколько компаний между собой, а также создавать подписки на автоматические уведомления.

Программа ориентирована на частных инвесторов и предназначена для поддержки принятия инвестиционных решений на основе фундаментального анализа.

2 Условия выполнения программы

Для работы программы необходимо наличие следующих программных и аппаратных средств:

- операционная система Windows 10 или более поздняя версия.
- установленный интерпретатор Python версии 3.10 и выше.
- установленная система управления базами данных PostgreSQL.
- подключение к сети Интернет для получения рыночных данных.
- современный веб-браузер (Google Chrome, Mozilla Firefox, Microsoft Edge или аналогичный).
- установленные библиотеки Python, используемые проектом.

Перед запуском программы необходимо убедиться в работоспособности сервера PostgreSQL и наличии настроенного подключения к базе данных.

3 Выполнение программы

1) Запуск приложения

- Предварительные условия: сервер PostgreSQL должен быть запущен и доступен для подключения.
- Действие: открыть терминал в каталоге проекта и выполнить команду:
`python invest_app.py`
- После запуска открыть в браузере адрес: `http://127.0.0.1:5000`
- Ожидаемый результат: в браузере отображается главная страница приложения.

2) Просмотр информации об акции

- Действие: выбрать интересующую компанию из списка доступных акций.
- Ожидаемый результат: отображаются текущая стоимость акции, график изменения выбранного показателя и основные финансовые характеристики компании.

3) Изменение периода отображения графика

- Действие: выбрать необходимый временной интервал отображения данных.
- Ожидаемый результат: график перестраивается в соответствии с выбранным периодом и отображает данные только за указанный промежуток времени.

4) Просмотр инвестиционных коэффициентов

- Действие: выбрать компанию и перейти к просмотру финансовых показателей.
- Ожидаемый результат: отображаются рассчитанные значения коэффициентов P/E, P/B и ROE, а также сведения о финансовой отчетности и дивидендных выплатах компании.

5) Сравнение нескольких компаний

- Действие: добавить одну или несколько компаний в список сравнения.

643.МГУЛ.090301-84 34 01

- Ожидаемый результат: на графике одновременно отображаются данные нескольких компаний, что позволяет выполнять сравнительный анализ выбранного показателя.
- 6) Создание подписки на уведомление
- Действие: указать адрес электронной почты, выбрать условие уведомления и задать требуемое значение показателя.
 - Ожидаемый результат: подписка сохраняется в базе данных и становится доступной для автоматической проверки системой.
- 7) Получение уведомления
- Действие: дождаться выполнения условий подписки.
 - Ожидаемый результат: на указанный адрес электронной почты поступает уведомление с информацией о выполнении заданных условий и ссылкой для управления подписками.
- 8) Управление подписками
- Действие: перейти по ссылке управления подписками, полученной в электронном письме.
 - Ожидаемый результат: отображается список активных подписок пользователя.
 - Действие: нажать кнопку удаления подписки.
 - Ожидаемый результат: выбранная подписка удаляется из базы данных и больше не участвует в проверке условий уведомлений.

4 Сообщения оператору

В процессе работы программы пользователь может столкнуться со следующими сообщениями и ошибками.

Возможные причины ошибок:

1. Не удалось подключиться к базе данных.
 - Причина: сервер PostgreSQL не запущен либо указаны неверные параметры подключения.
 - Рекомендуемое действие: проверить настройки подключения и работоспособность сервера базы данных.
2. Неверный адрес электронной почты.
 - Причина: адрес введен некорректно либо содержит ошибки.
 - Рекомендуемое действие: проверить правильность введенного адреса.
3. Неверная ссылка управления подписками.
 - Причина: ссылка содержит неверный или устаревший токен.
 - Рекомендуемое действие: использовать актуальную ссылку, полученную в уведомлении.
4. Подписка не найдена.
 - Причина: подписка была ранее удалена либо отсутствует в базе данных.
 - Рекомендуемое действие: создать новую подписку.

Рекомендации оператору:

- регулярно контролировать работоспособность сервера базы данных.
- обеспечивать стабильное подключение к сети Интернет.
- контролировать свободное место для хранения исторических рыночных данных.
- выполнять резервное копирование базы данных.

[illegible]